



**Visual Recognition using Deep Learning**

# **Transformers for Object Recognition**

林彥宇 教授

Yen-Yu Lin, Professor

國立陽明交通大學 資訊工程學系

Computer Science, National Yang Ming Chiao Tung University

Some slides are modified from Hung-yi Lee, Shusen Wang, and Sy-Kai Chen

# Outline

- Transformer
- ViT
- DeiT
- Swin Transformer



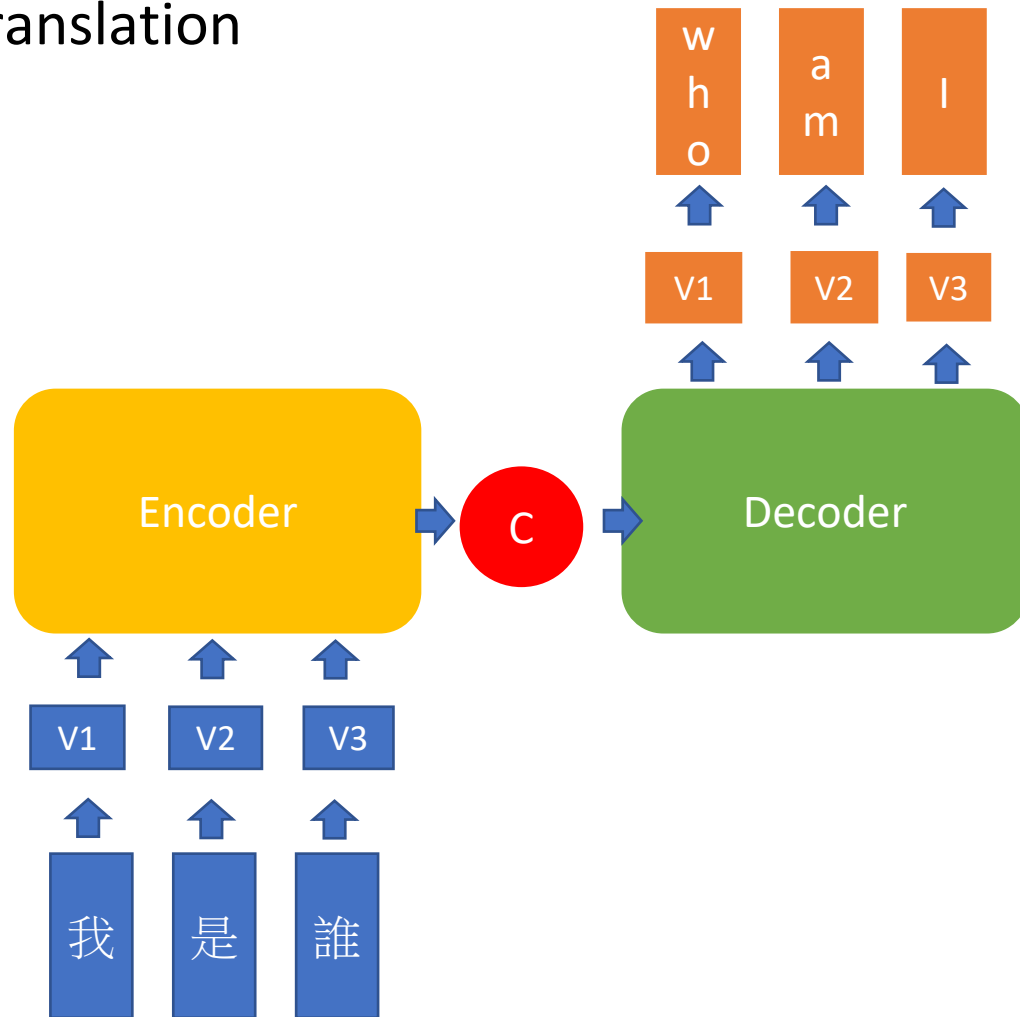
# Outline

- Transformer
  - Most slides are (modified) from Prof. Hung-yi Lee
- ViT
- DeiT
- Swin Transformer

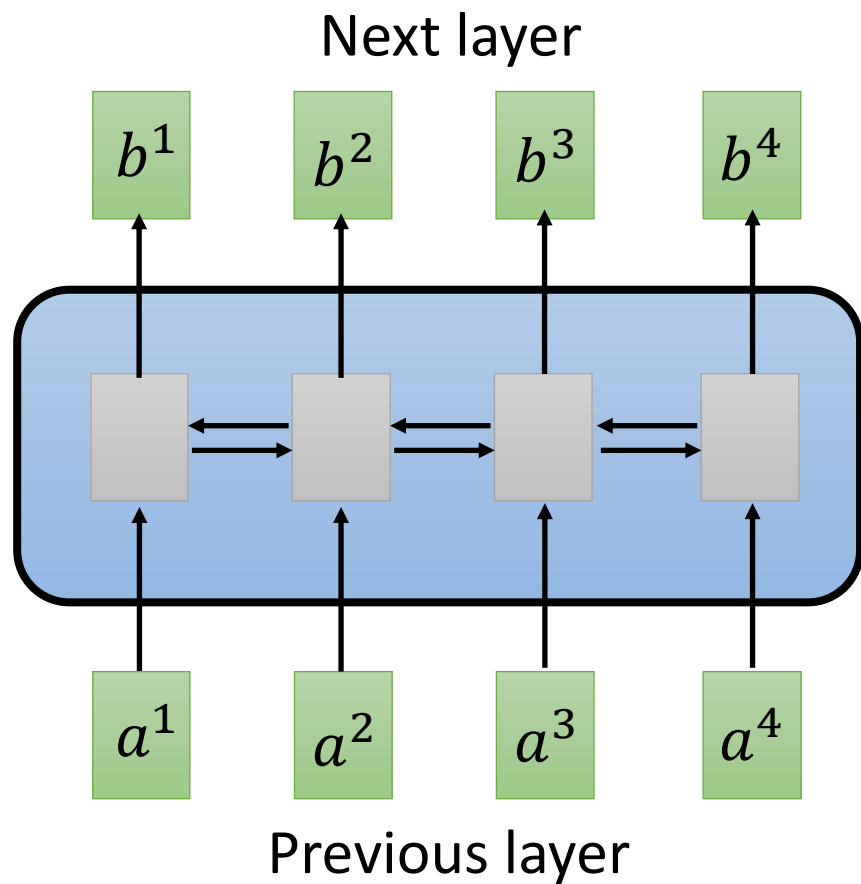


# Seq2seq model

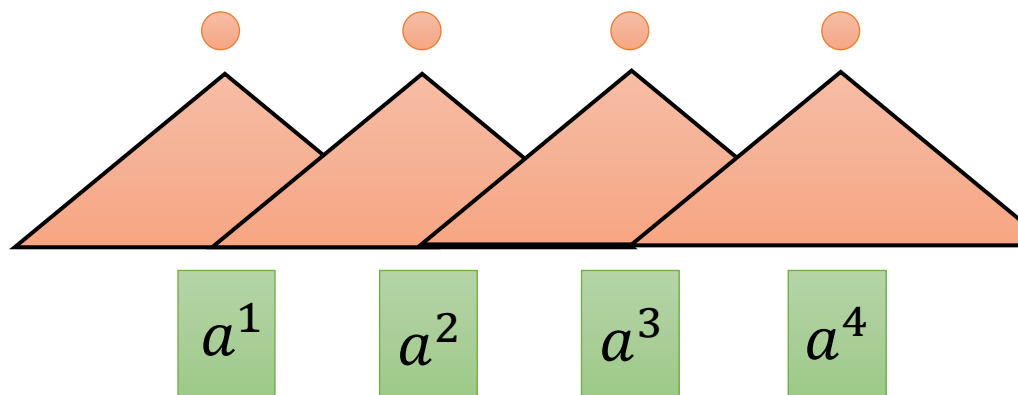
- Language translation



# Sequence

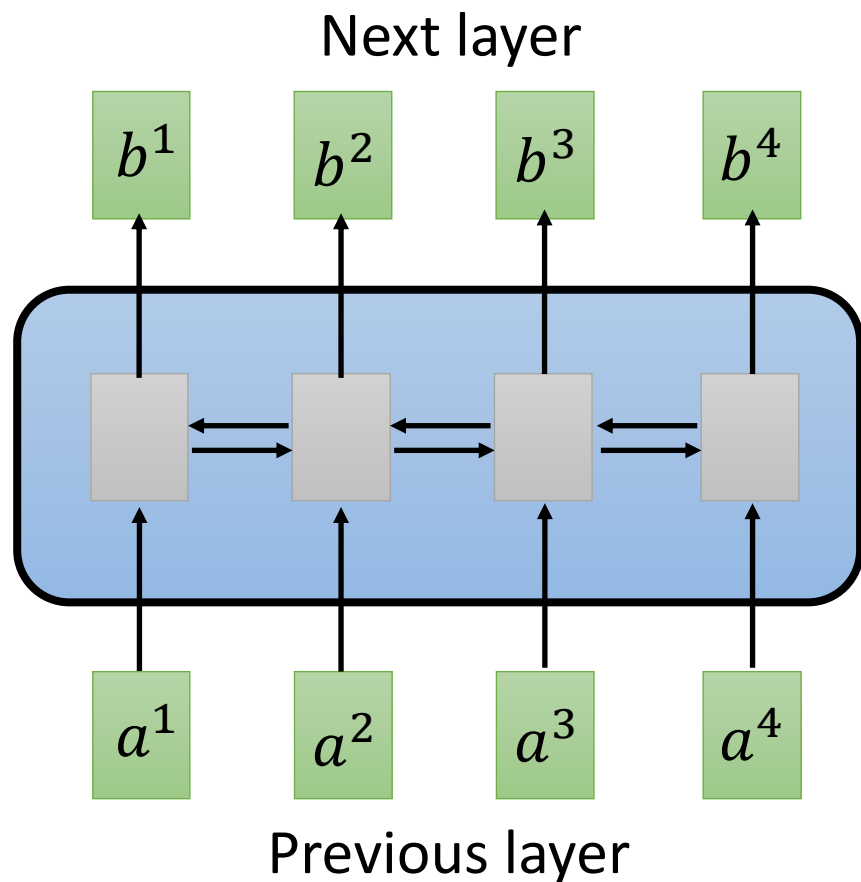


Hard to parallel !



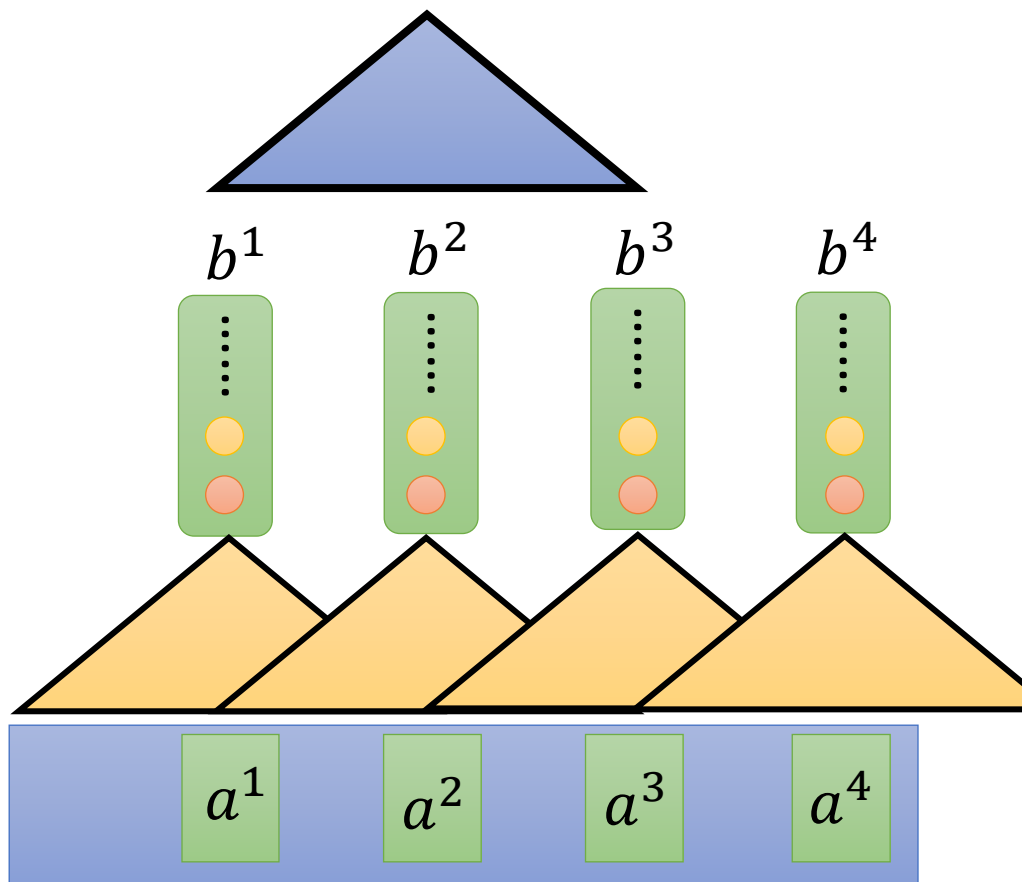
Using CNN to replace RNN

## Sequence



Hard to parallel

Filters in higher layer can consider longer sequence



Using CNN to replace RNN  
(CNN can parallel)

# Self-attention

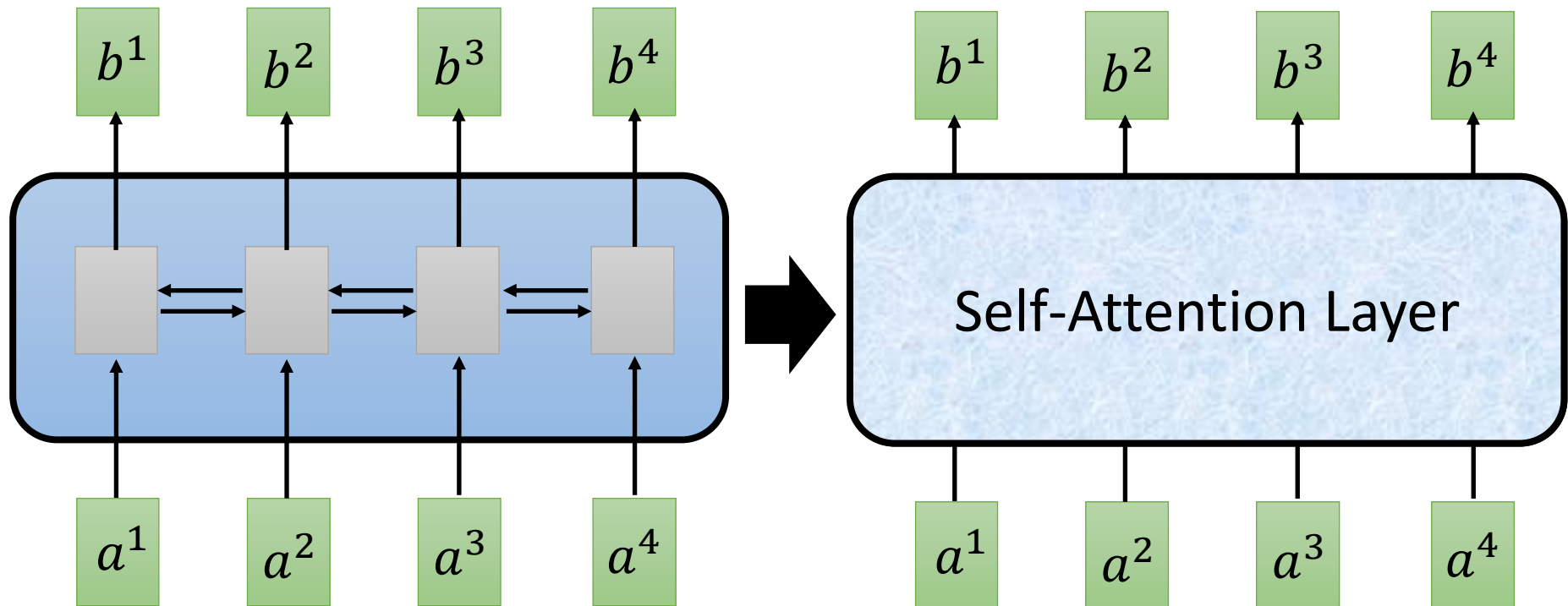
- What is self attention?
  - Self attention is an attention mechanism relating **different positions of a single sequence** in order to compute a representation of the sequence
- Why self attention?
  - Solve the fundamental limitation in recurrent models



# Self-Attention

$b^i$  is obtained based on the whole input sequence.

$b^1, b^2, b^3$ , and  $b^4$  can be parallelly computed.





# Self-attention

<https://arxiv.org/abs/1706.03762>

$q$ : query (to match others)

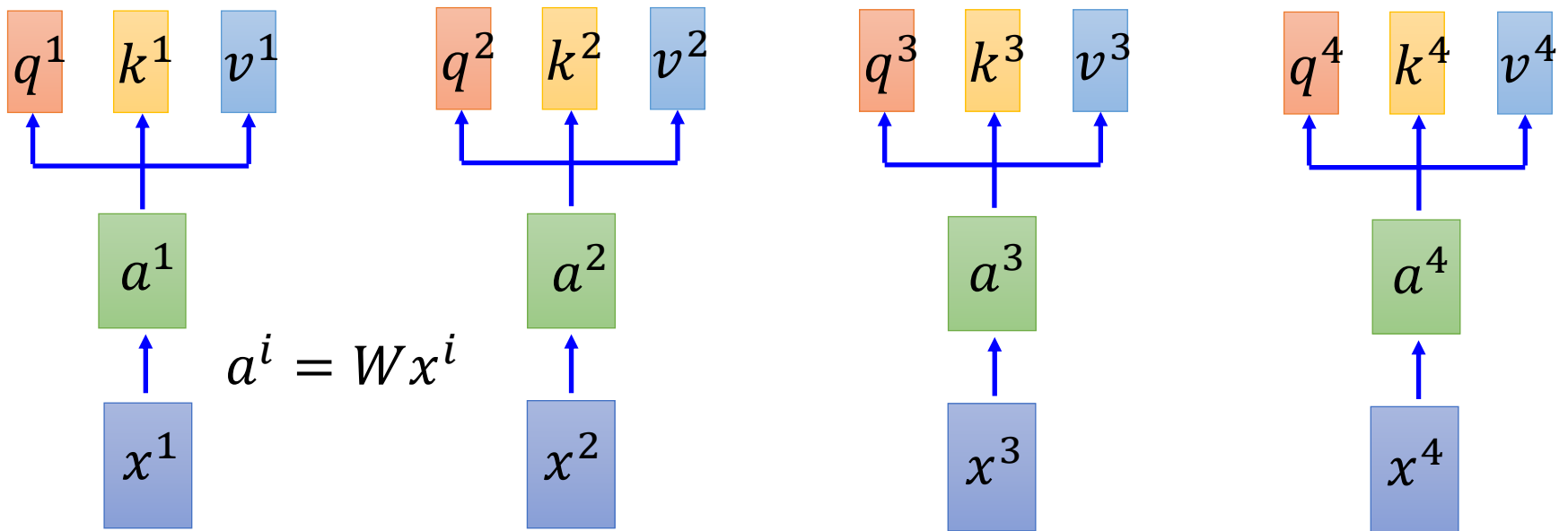
$$q^i = W^q a^i$$

$k$ : key (to be matched)

$$k^i = W^k a^i$$

$v$ : value (information to be extracted)

$$v^i = W^v a^i$$

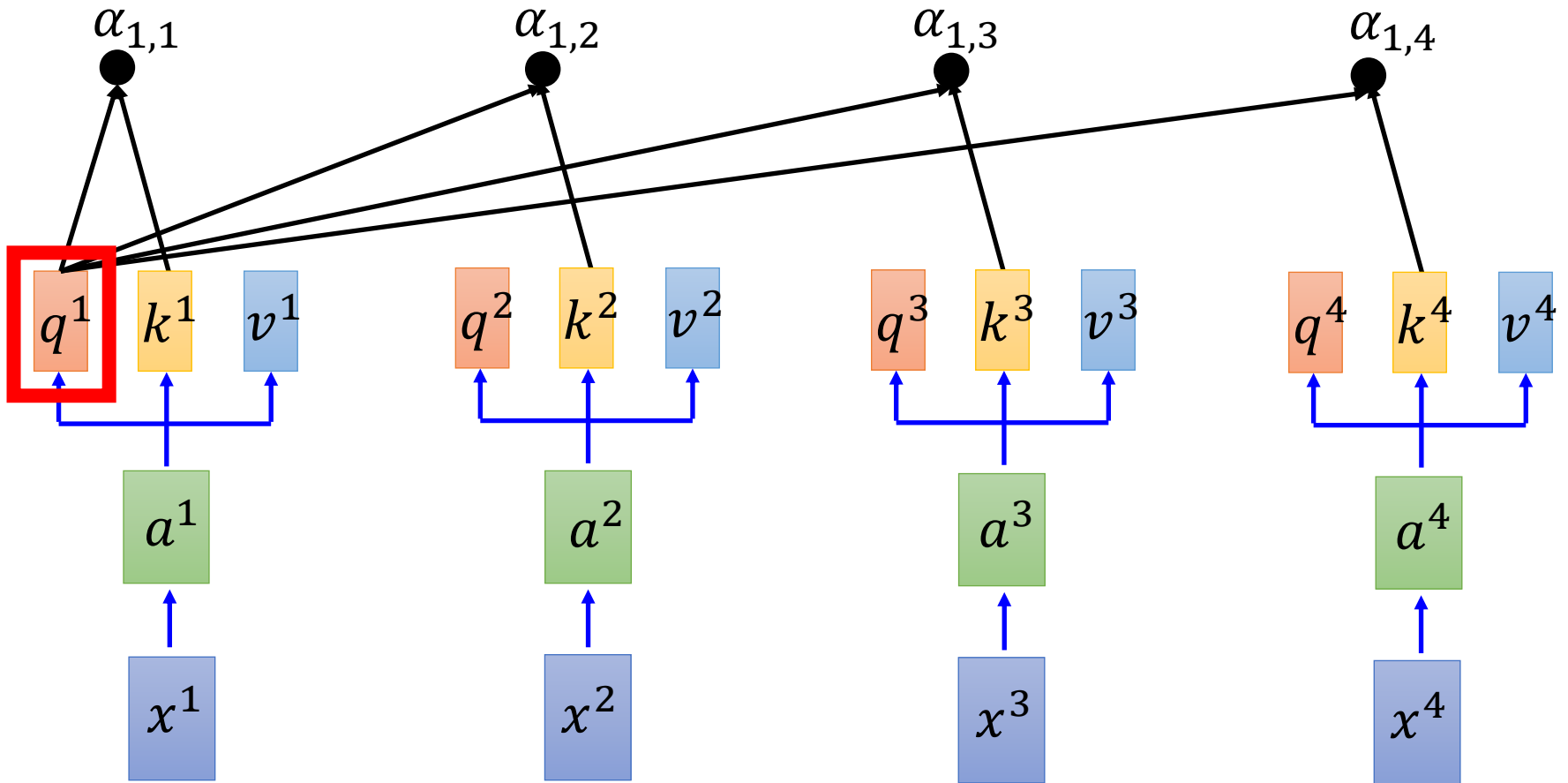


# Self-attention

Calculate attention between each query and all keys

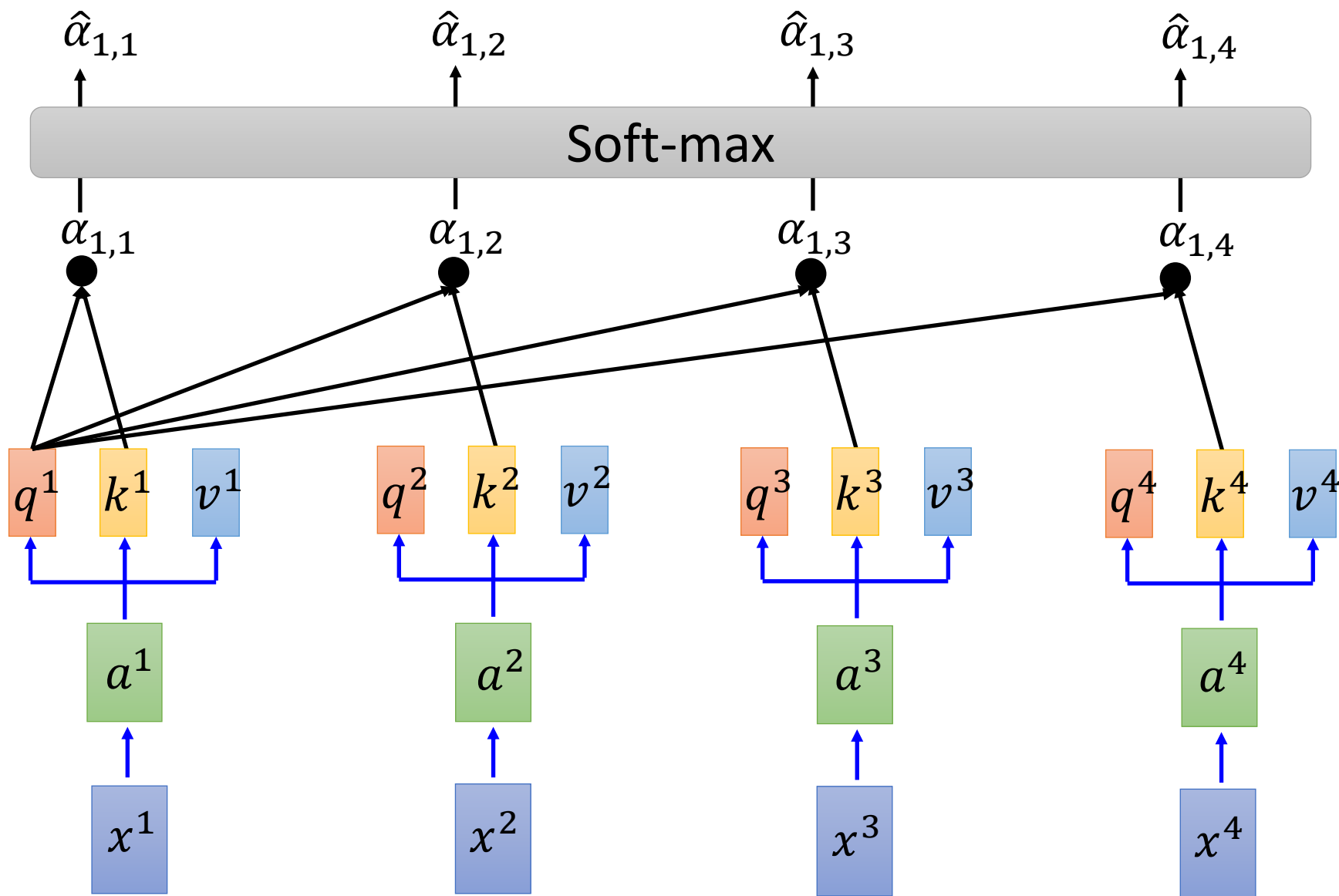
$d$  is the dim of  $q$  and  $k$

Scaled Dot-Product Attention:  $\alpha_{1,i} = \underbrace{q^1 \cdot k^i}_{\text{dot product}} / \sqrt{d}$



# Self-attention

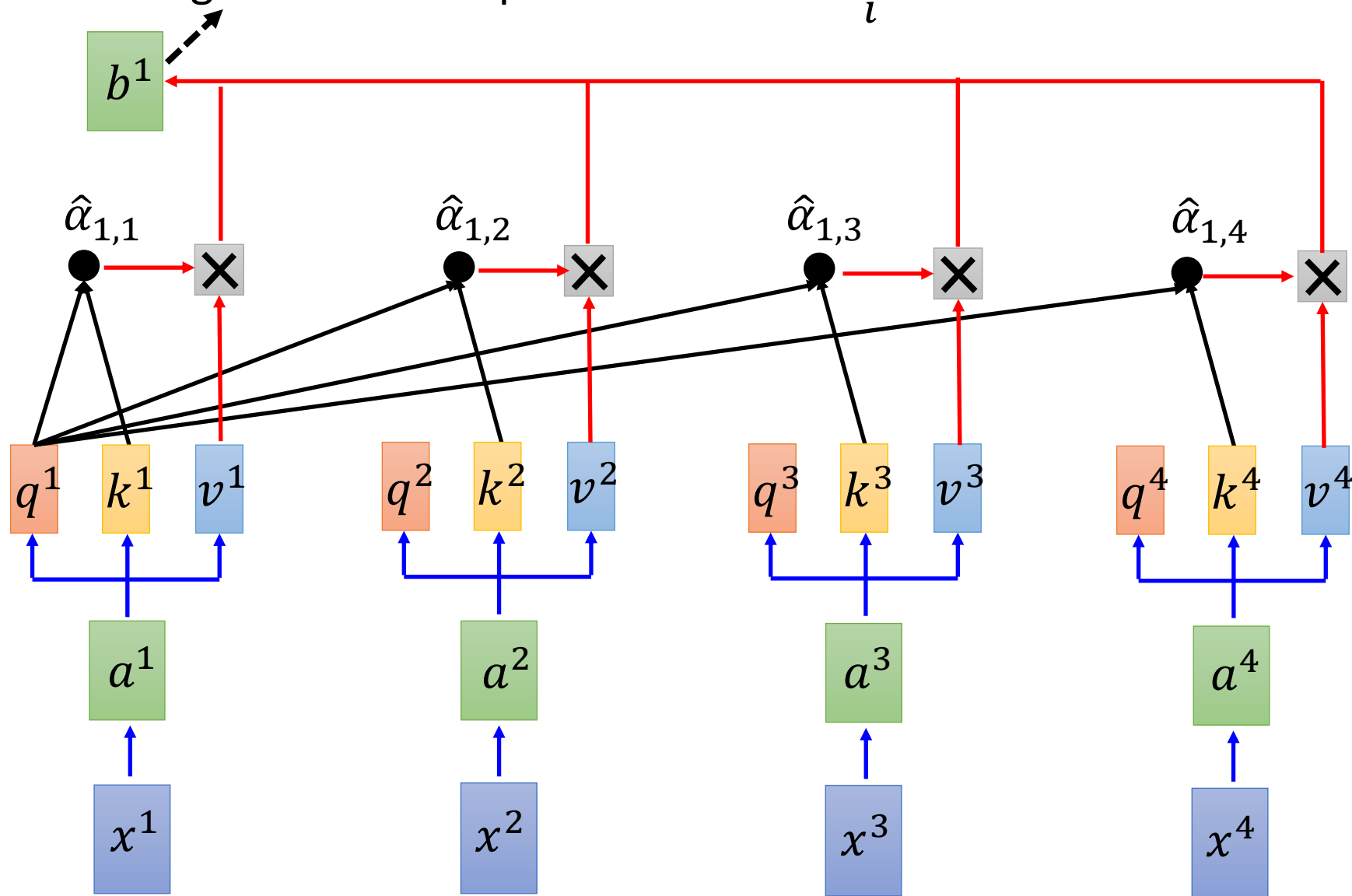
$$\hat{\alpha}_{1,i} = \exp(\alpha_{1,i}) / \sum_j \exp(\alpha_{1,j})$$



# Self-attention

Considering the whole sequence

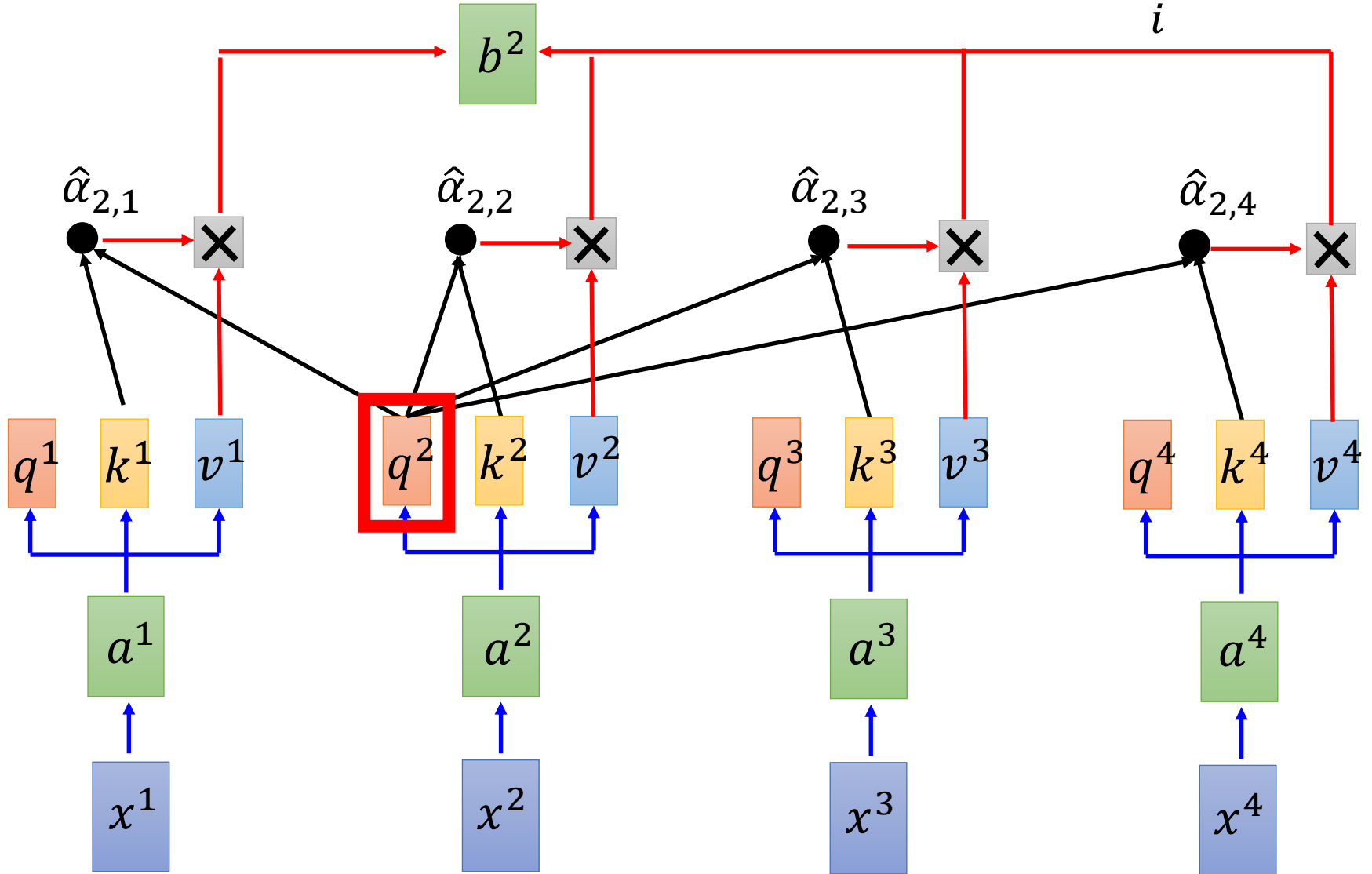
$$b^1 = \sum_i \hat{\alpha}_{1,i} v^i$$



# Self-attention

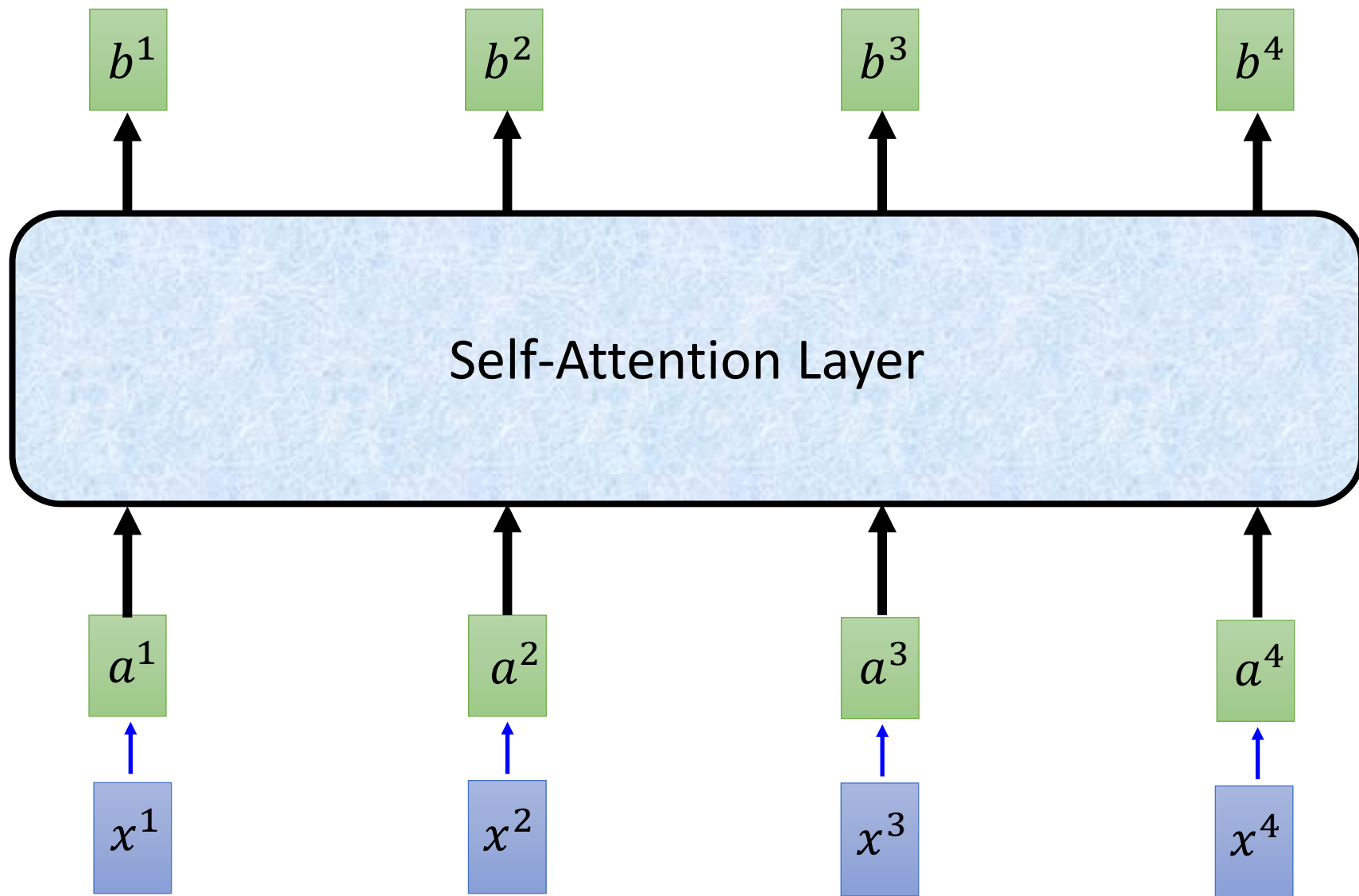
Calculate attention between each query and all keys

$$b^2 = \sum_i \hat{\alpha}_{2,i} v^i$$



# Self-attention

$b^1, b^2, b^3$ , and  $b^4$  can be parallelly computed.



# Self-attention

$$q^i = W^q a^i$$

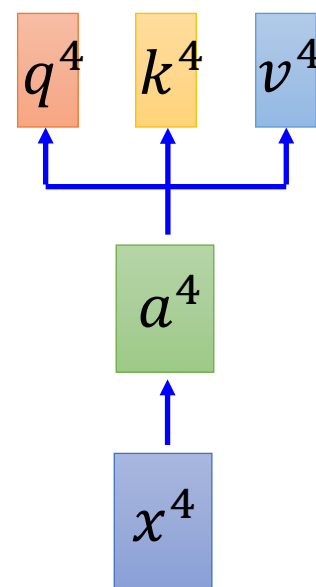
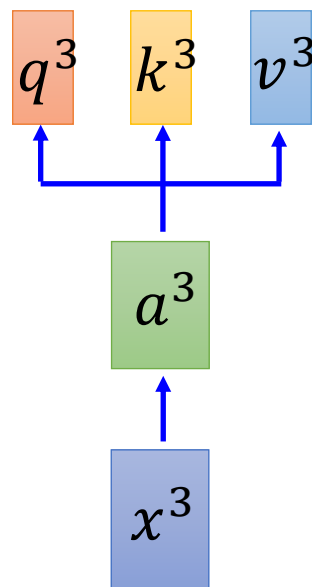
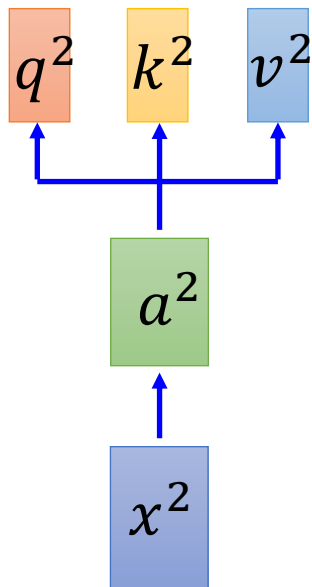
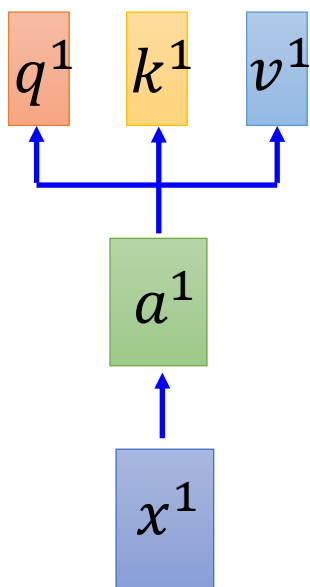
$$k^i = W^k a^i$$

$$v^i = W^v a^i$$

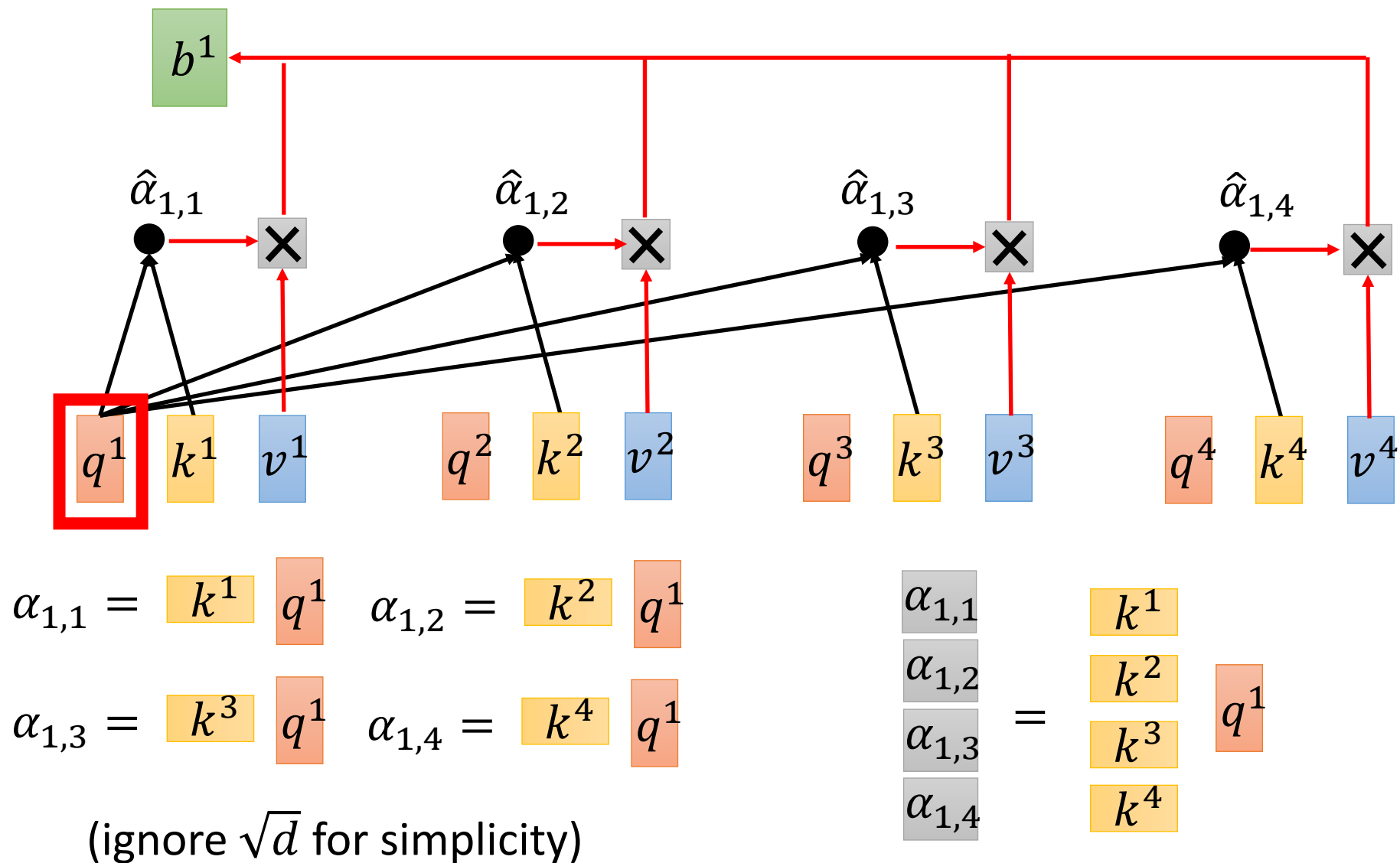
$$\begin{matrix} q^1 & q^2 & q^3 & q^4 \\ Q \end{matrix} = \begin{matrix} W^q & \begin{matrix} a^1 & a^2 & a^3 & a^4 \\ I \end{matrix} \end{matrix}$$

$$\begin{matrix} k^1 & k^2 & k^3 & k^4 \\ K \end{matrix} = \begin{matrix} W^k & \begin{matrix} a^1 & a^2 & a^3 & a^4 \\ I \end{matrix} \end{matrix}$$

$$\begin{matrix} v^1 & v^2 & v^3 & v^4 \\ V \end{matrix} = \begin{matrix} W^v & \begin{matrix} a^1 & a^2 & a^3 & a^4 \\ I \end{matrix} \end{matrix}$$



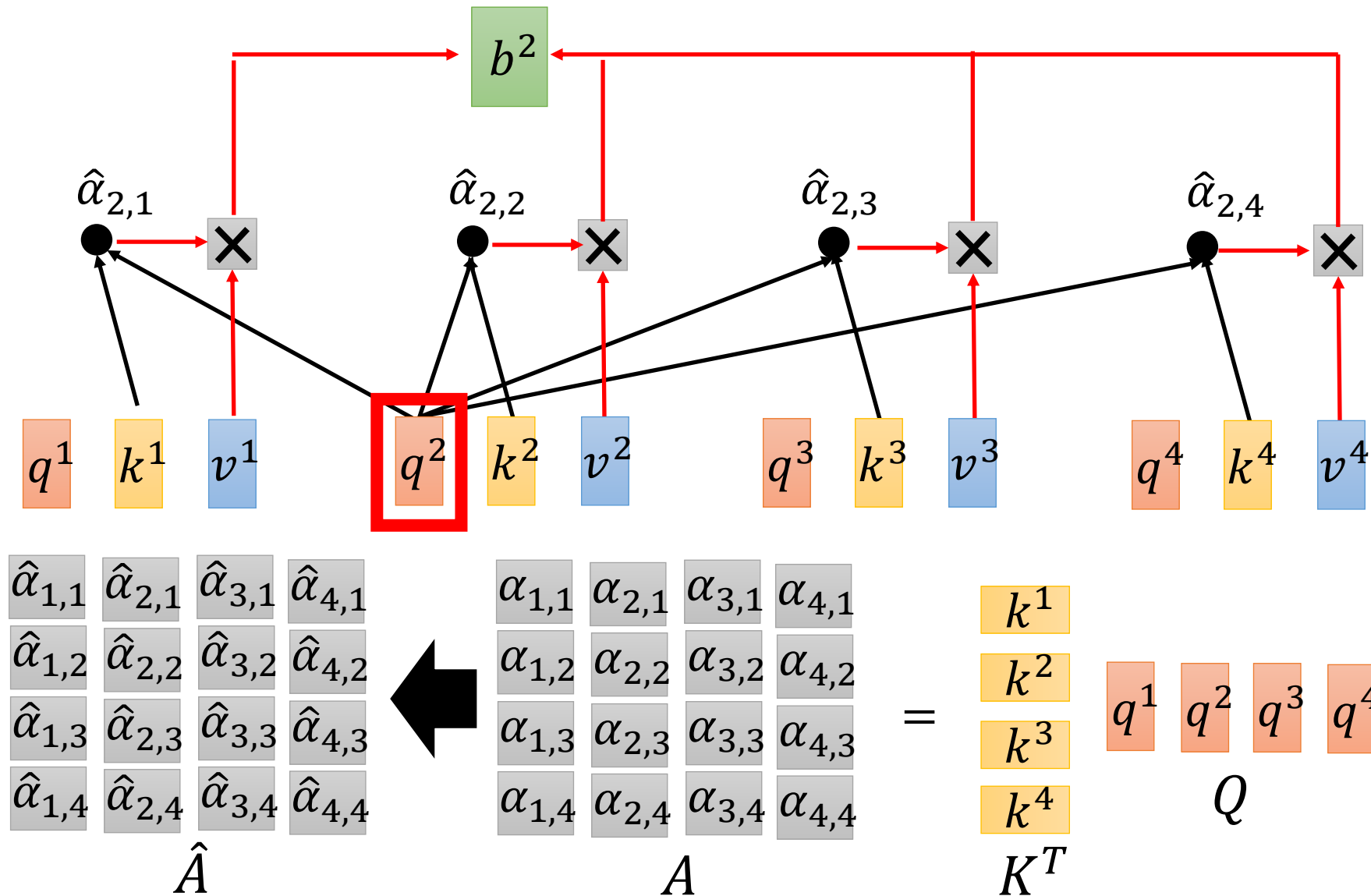
# Self-attention





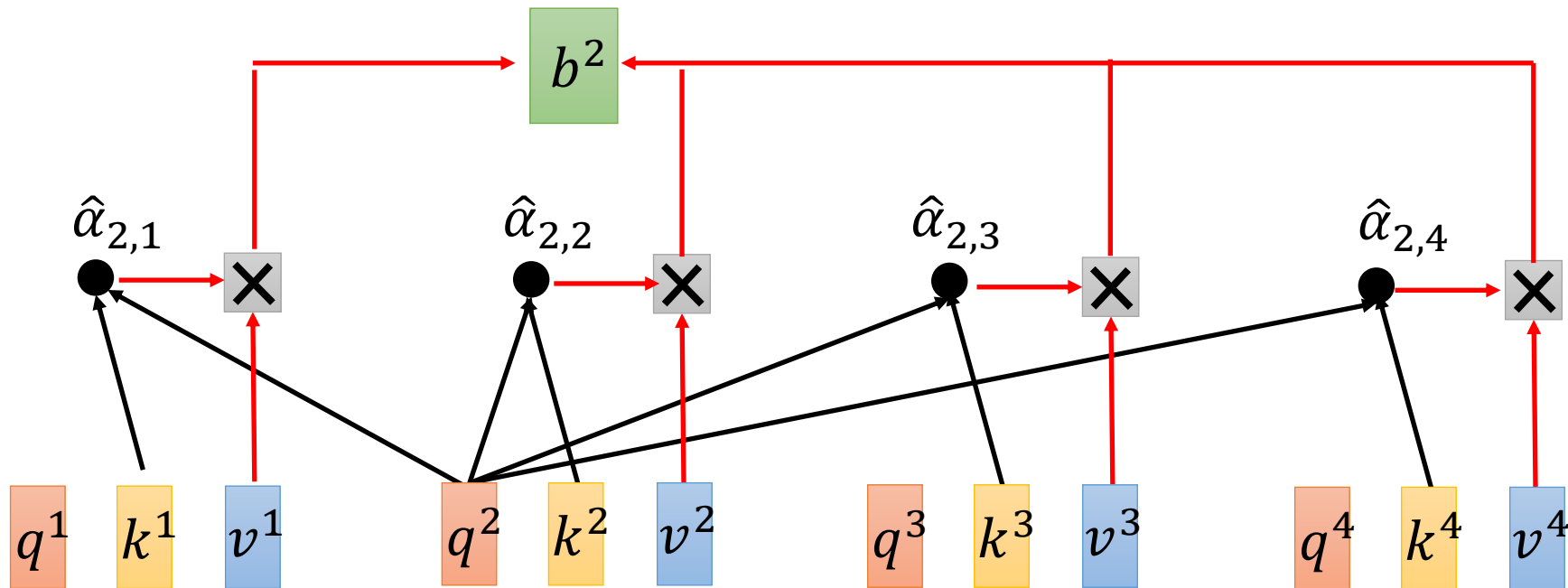
# Self-attention

$$b^2 = \sum_i \hat{\alpha}_{2,i} v^i$$



# Self-attention

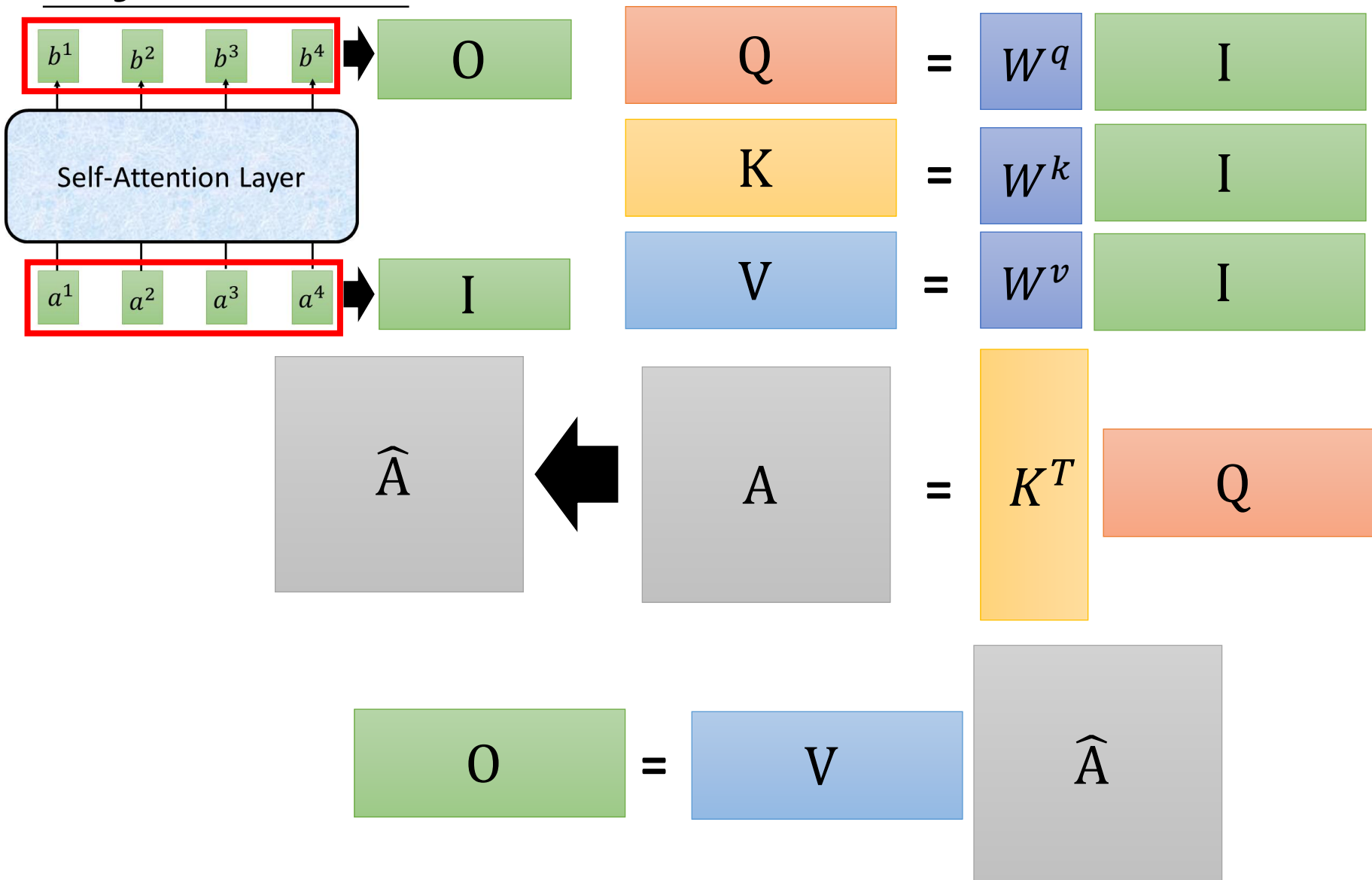
$$b^2 = \sum_i \hat{\alpha}_{2,i} v^i$$



$$\begin{matrix} b^1 & b^2 & b^3 & b^4 \\ \hline \end{matrix} \quad \begin{matrix} = \\ \end{matrix} \quad \begin{matrix} v^1 & v^2 & v^3 & v^4 \\ \hline \end{matrix} \quad \begin{matrix} \hat{\alpha}_{1,1} & \hat{\alpha}_{2,1} & \hat{\alpha}_{3,1} & \hat{\alpha}_{4,1} \\ \hat{\alpha}_{1,2} & \hat{\alpha}_{2,2} & \hat{\alpha}_{3,2} & \hat{\alpha}_{4,2} \\ \hat{\alpha}_{1,3} & \hat{\alpha}_{2,3} & \hat{\alpha}_{3,3} & \hat{\alpha}_{4,3} \\ \hat{\alpha}_{1,4} & \hat{\alpha}_{2,4} & \hat{\alpha}_{3,4} & \hat{\alpha}_{4,4} \end{matrix}$$

$\hat{A}$

# Self-attention



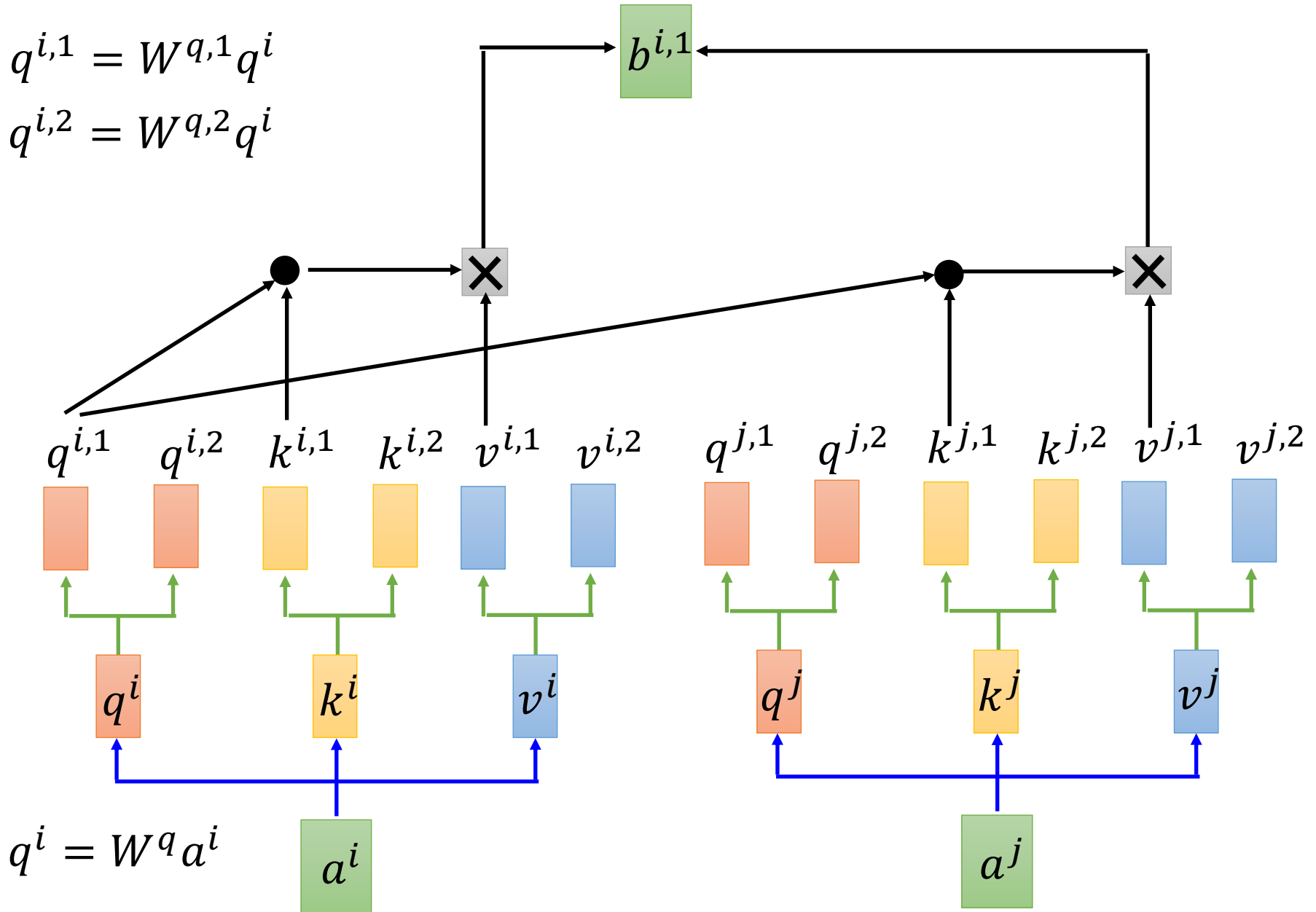
GPU can accelerate matrix multiplications

# Multi-head Self-attention

(2 heads as example)

$$q^{i,1} = W^{q,1} q^i$$

$$q^{i,2} = W^{q,2} q^i$$

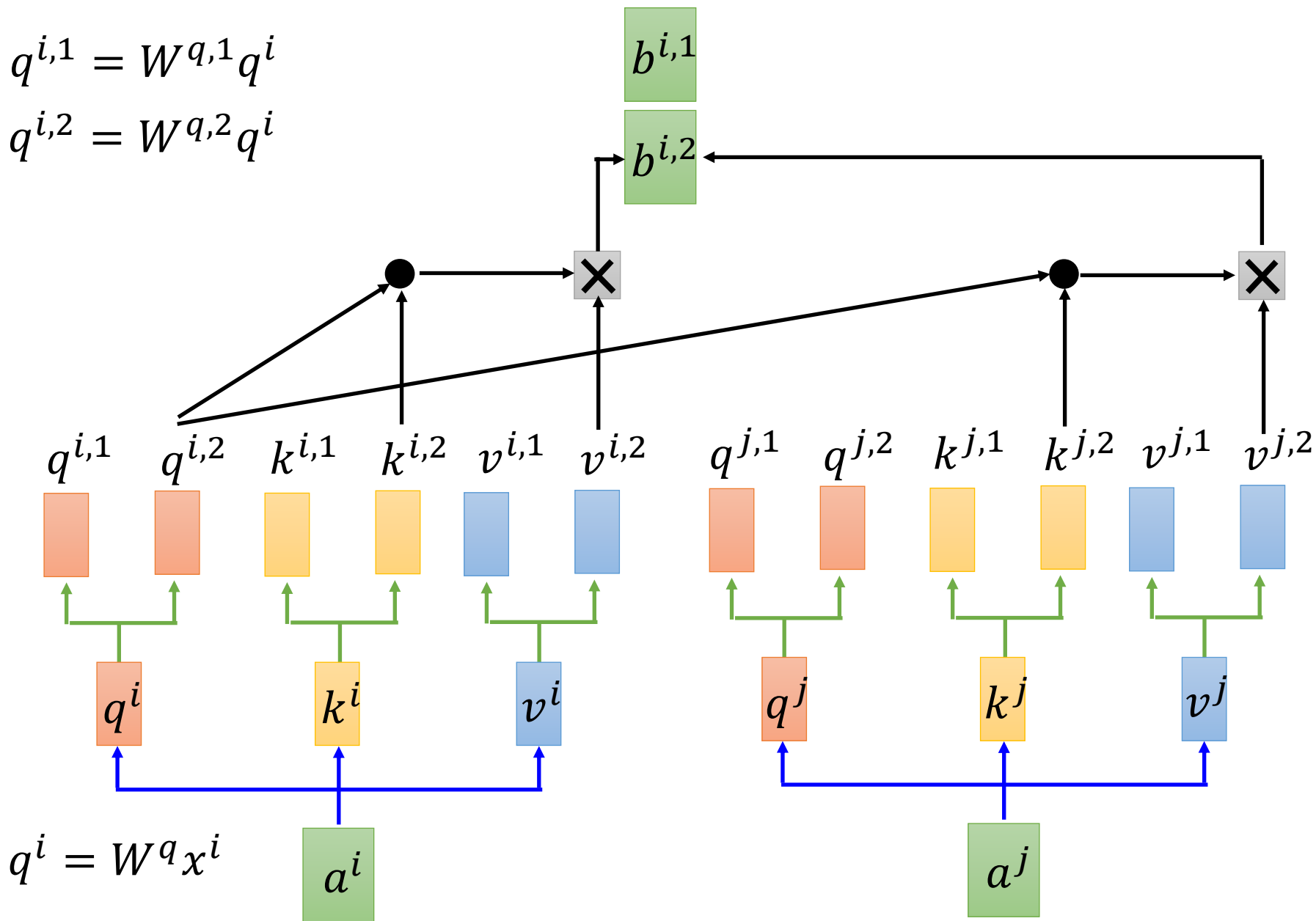


# Multi-head Self-attention

(2 heads as example)

$$q^{i,1} = W^{q,1} q^i$$

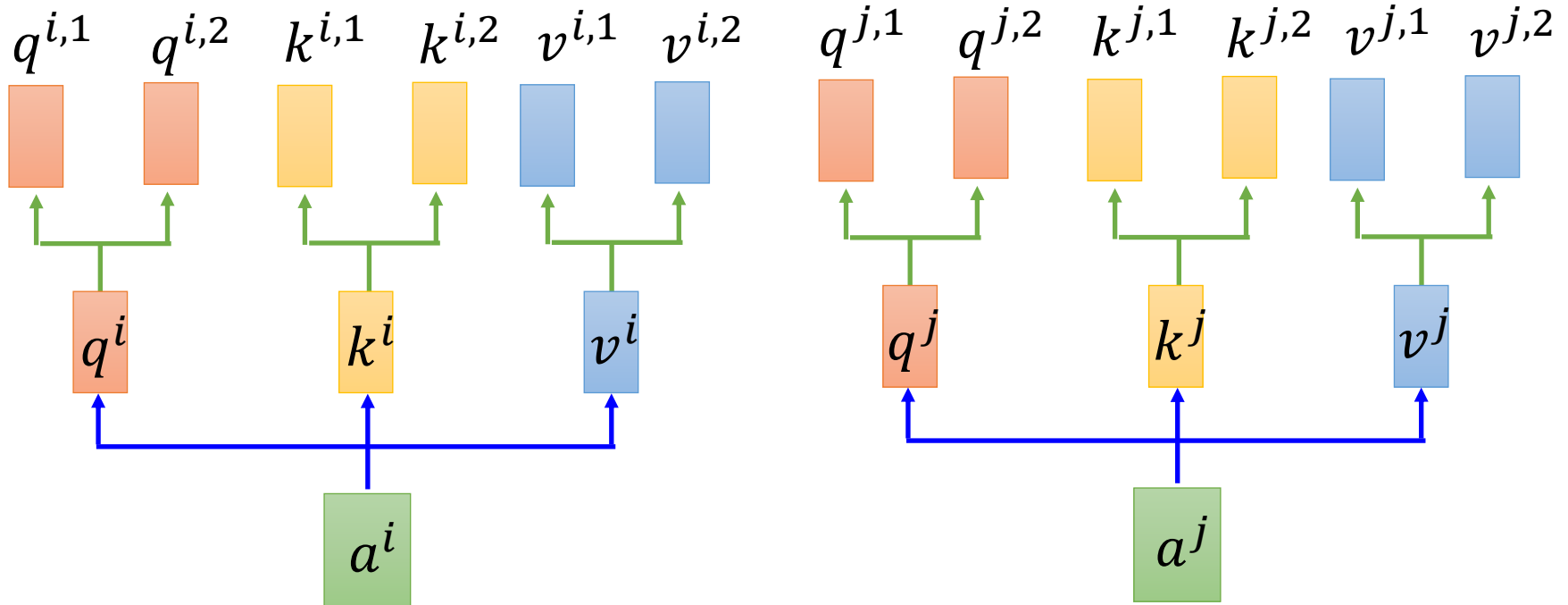
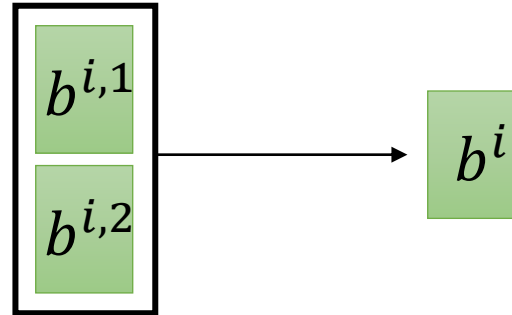
$$q^{i,2} = W^{q,2} q^i$$



# Multi-head Self-attention

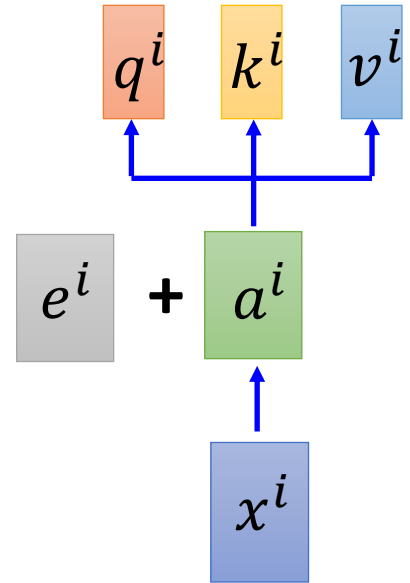
(2 heads as example)

$$b^i = W^O \begin{bmatrix} b^{i,1} \\ b^{i,2} \end{bmatrix}$$

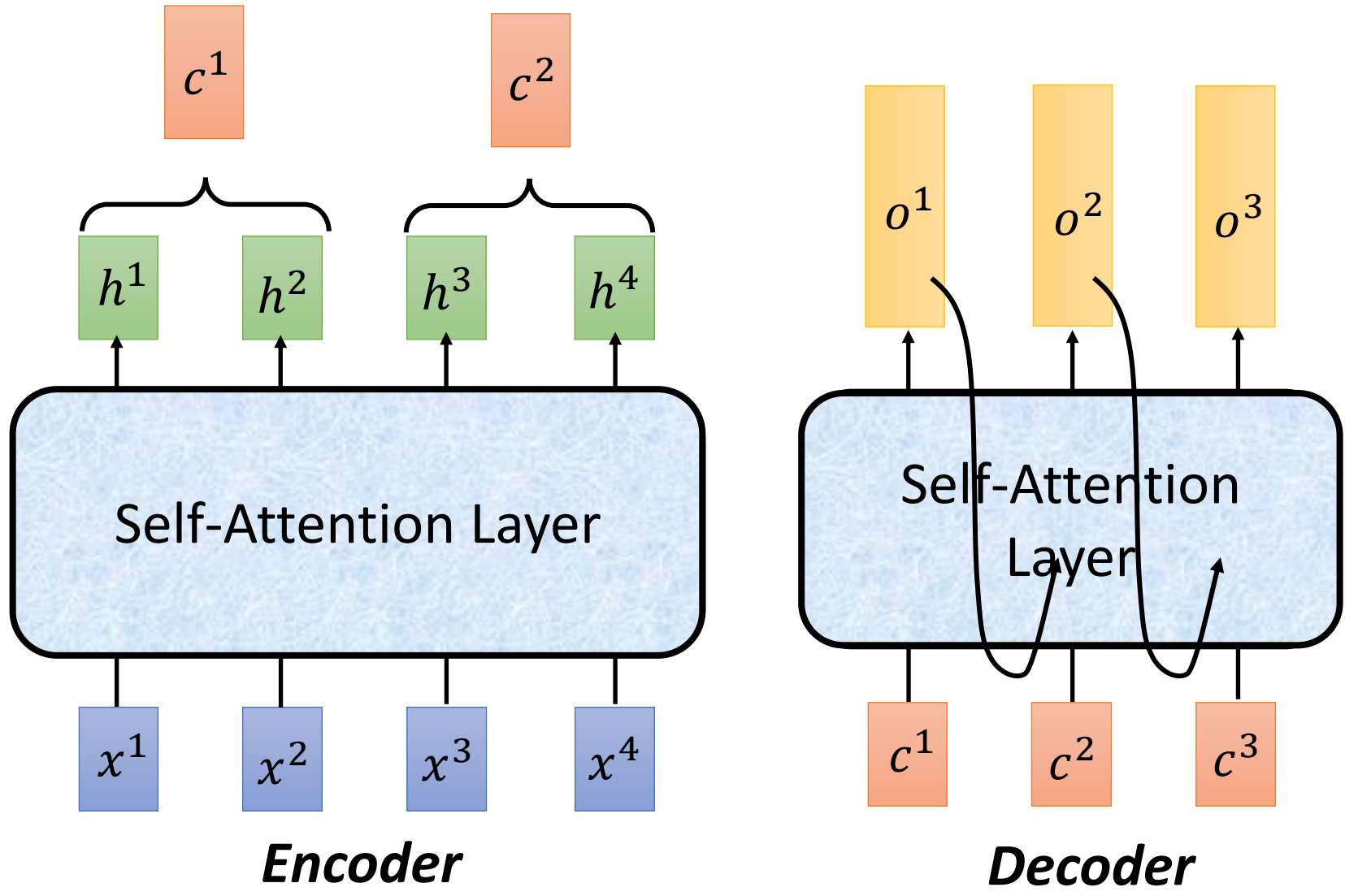


# Positional Encoding

- No position information in self-attention.
- Original paper: each position has a unique positional vector  $e^i$  (not learned from data)



# Seq2seq with Attention

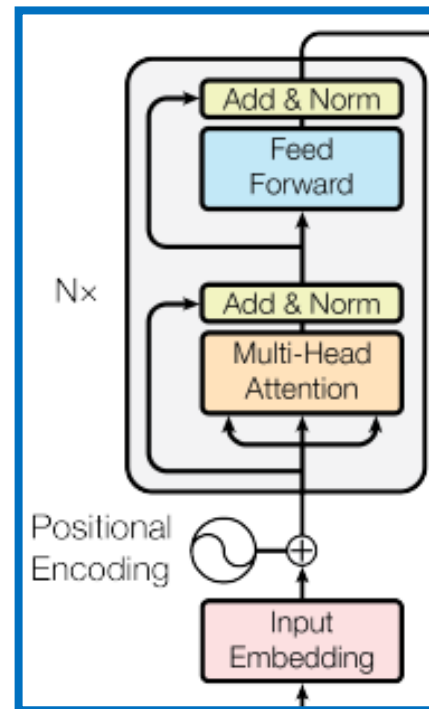




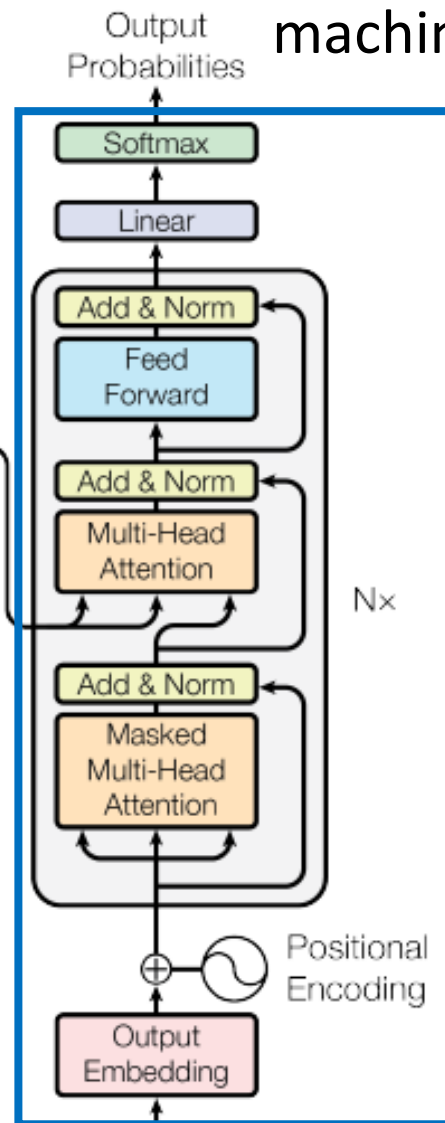
# Transformer

Using Chinese to English translation as example

Encoder



機器學習

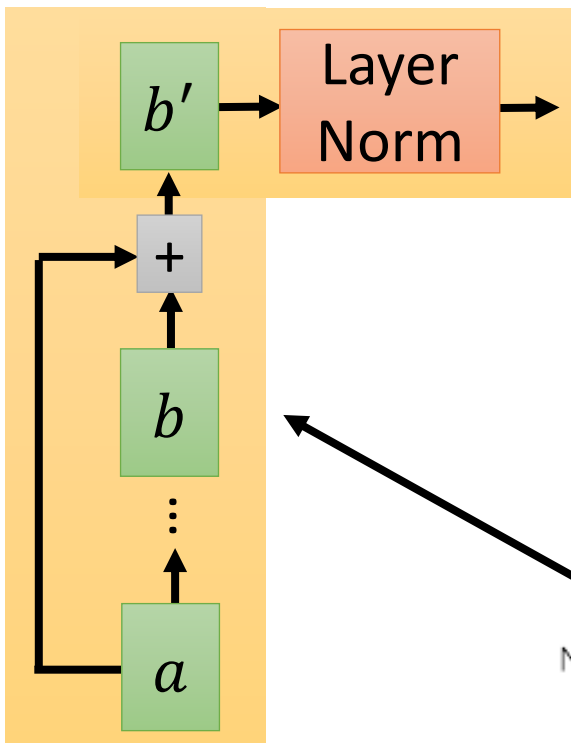


Output Probabilities machine learning

<BOS>

machine

# Transformer

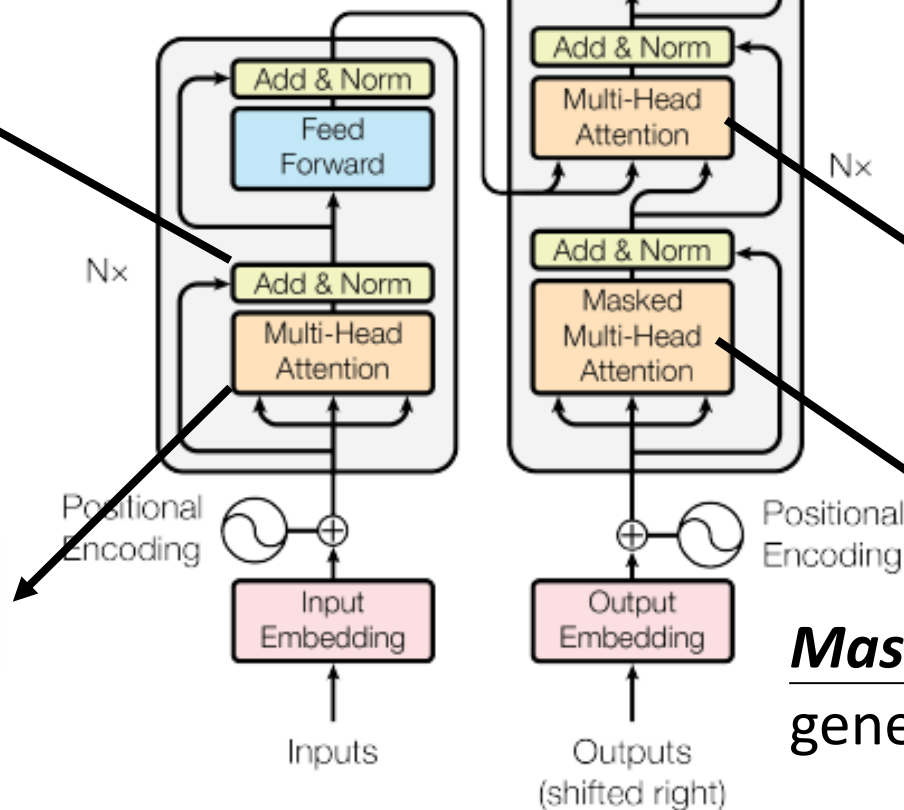
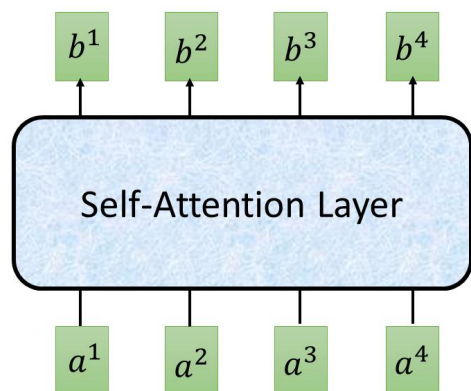


Layer Norm:

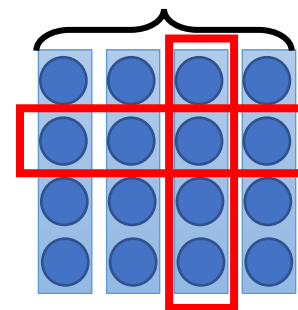
<https://arxiv.org/abs/1607.06450>

Batch Norm:

<https://www.youtube.com/watch?v=BZh1ltr5Rkg>



Batch Size



$\mu = 0,$   
 $\sigma = 1$   
Batch

$\mu = 0, \sigma = 1$   
Layer

attend on the  
input sequence

**Masked**: attend on the  
generated sequence

# Outline

- Transformer
- ViT
- DeiT
- Swin Transformer



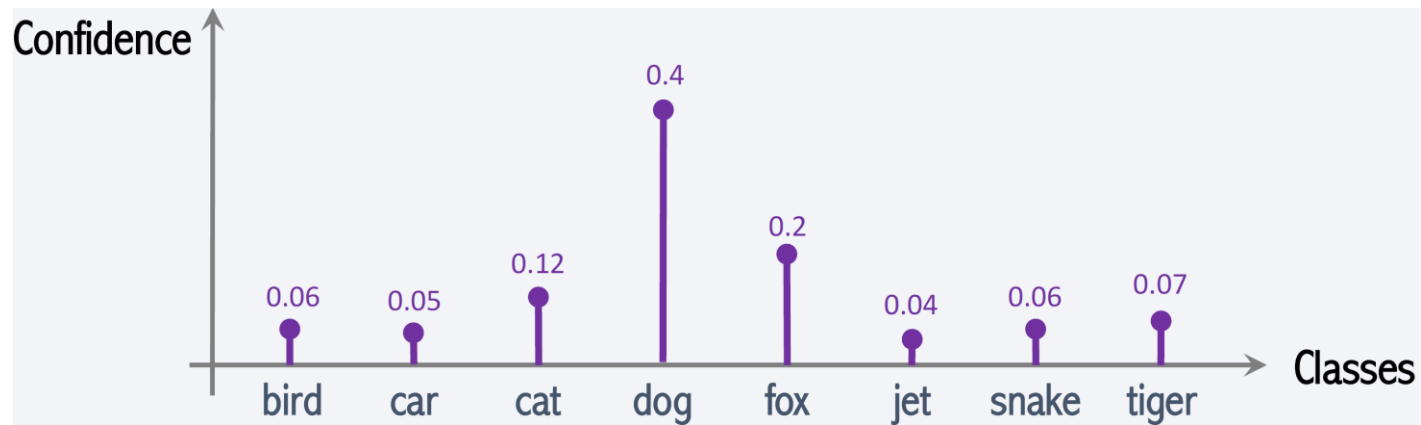
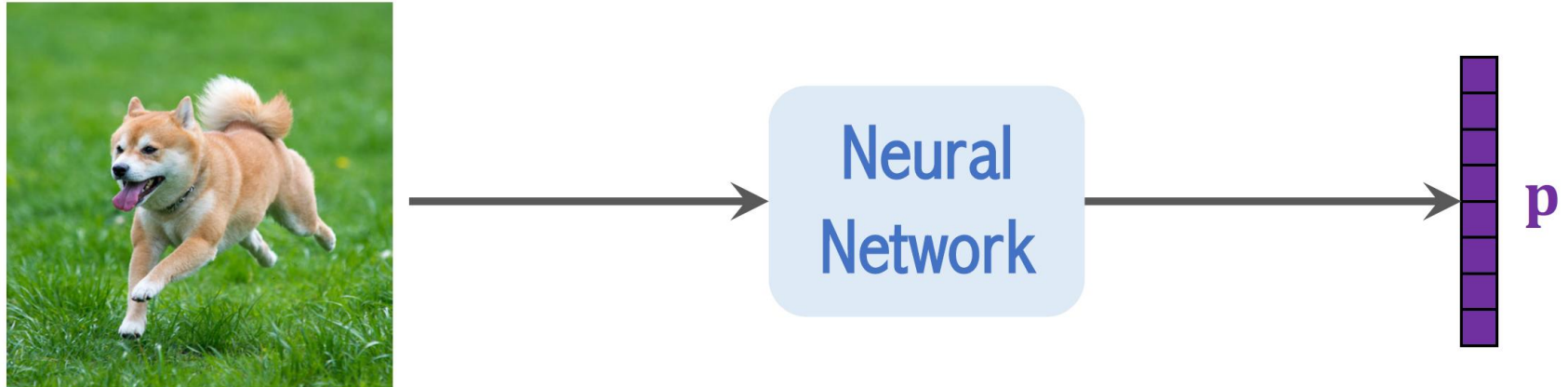
# Vision Transformer (ViT)

[Dosovitskiy et al., ICLR'21]

- Self-attention-based architectures, e.g., Transformers, have become the model of choice in natural language processing
- CNNs, e.g., ResNet, were the state-of-the-art models for object recognition
- Vision transformer (ViT) beats CNNs by a small margin, if the dataset for pre-training is sufficiently large
- ViT is developed mainly based on Transformer
  - Encoder: linear embedding, multi-head attention, layer normalization, ...
  - Position encoding

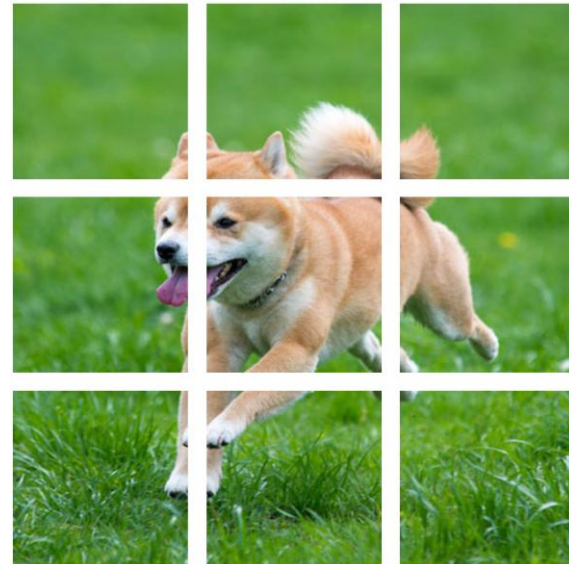
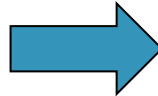


# Object recognition



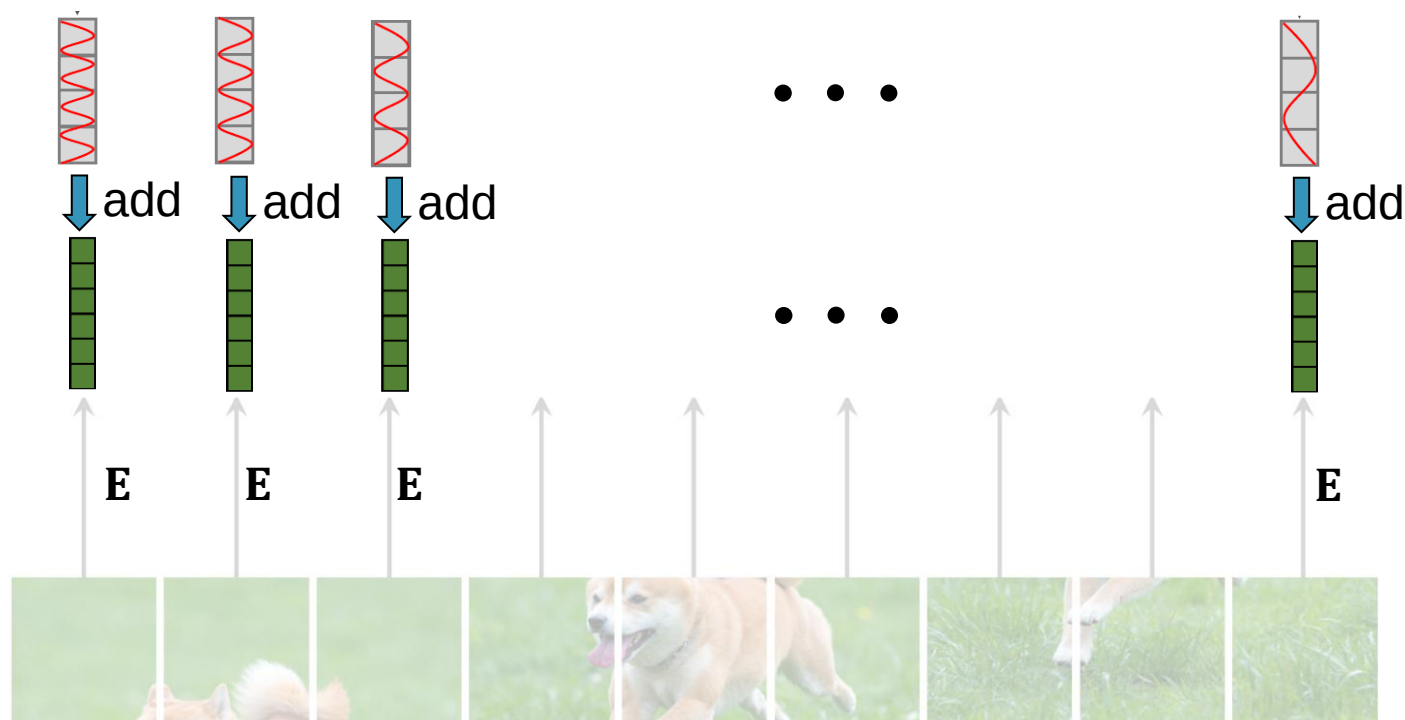
# An image as a set of patches

- To apply Transformer to object recognition with minor modifications, we split an image into patches
- Patches are specified by the user
  - Overlapping or non-overlapping
  - Patch size



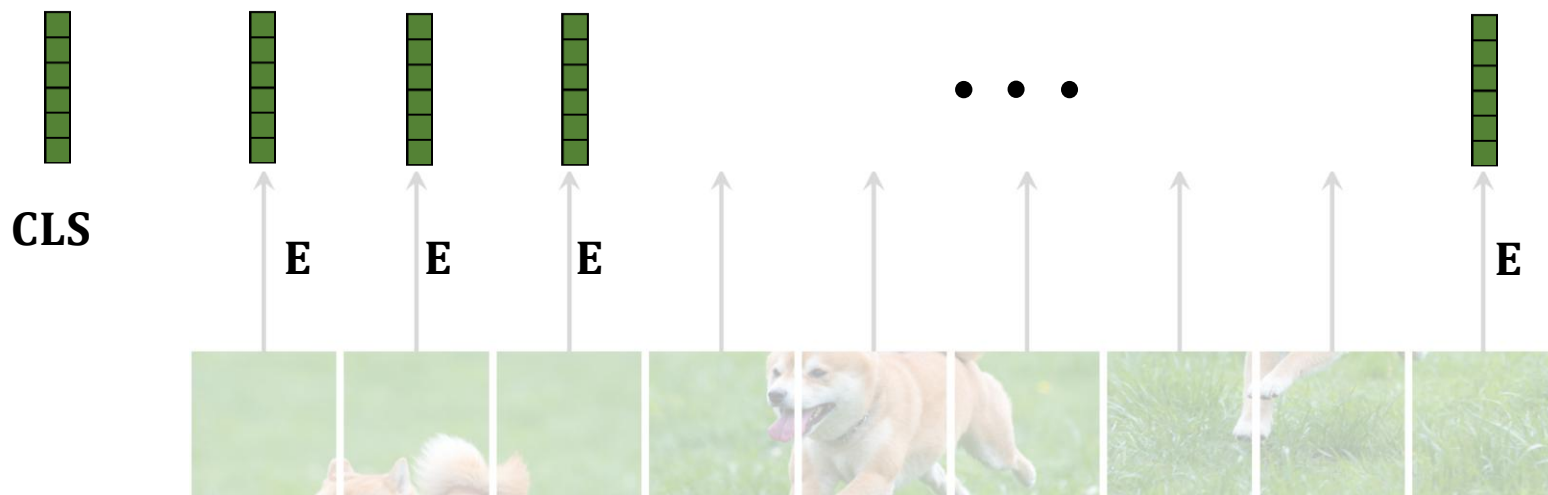
# Linear embeddings and positional encoding of patches

- We embed each patch by linear projection  $E$
- 2D learnable positional encoding is used and added to linear embedding



# Class token

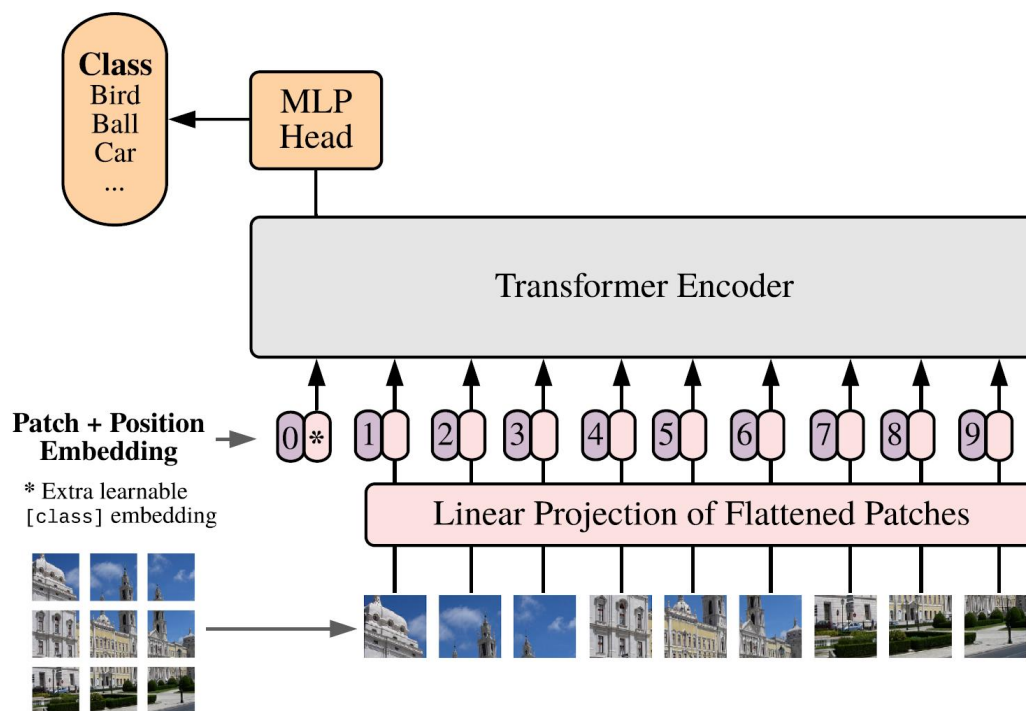
- A special token, called **class token**, is included
- Class token does not belong to any patch
- It is represented by a vector that is learnable
- During N blocks, it gathers information from other tokens, and its output vector is used to derive a classifier



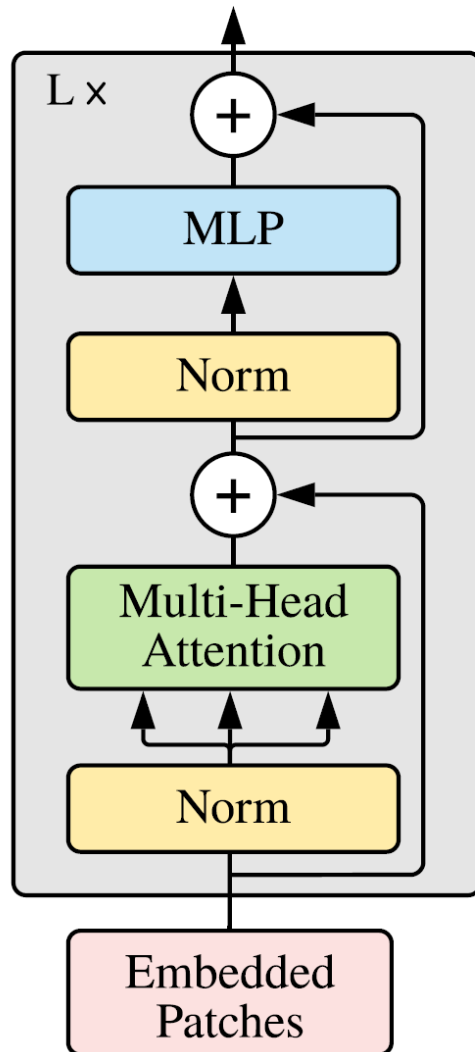


# Overview

1. Split an image into fixed-size patches (16 x 16), linearly embed each of them, and then add CLS token (learnable)
2. Add positional encoding
3. Transformer encoder operation
4. Use [cls] token to do the classification



# Transformer encoder



$$\mathbf{z}_0 = [\mathbf{x}_{\text{class}}; \mathbf{x}_p^1 \mathbf{E}; \mathbf{x}_p^2 \mathbf{E}; \dots; \mathbf{x}_p^N \mathbf{E}] + \mathbf{E}_{\text{pos}},$$

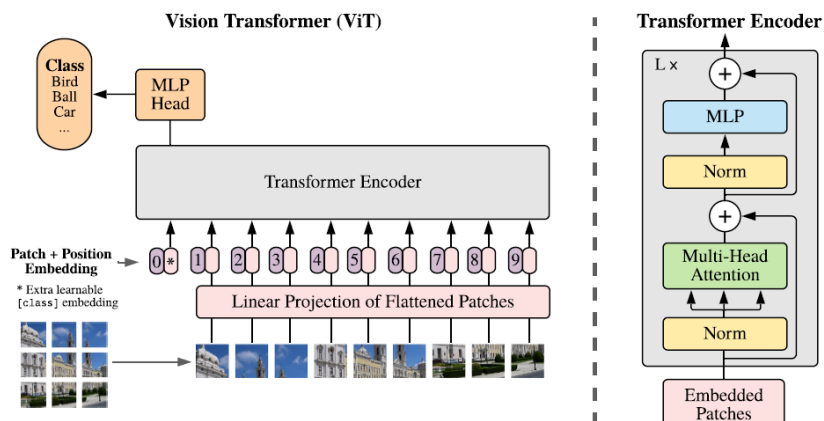
$$\mathbf{z}'_{\ell} = \text{MSA}(\text{LN}(\mathbf{z}_{\ell-1})) + \mathbf{z}_{\ell-1},$$

$$\mathbf{z}_{\ell} = \text{MLP}(\text{LN}(\mathbf{z}'_{\ell})) + \mathbf{z}'_{\ell},$$

$$\mathbf{y} = \text{LN}(\mathbf{z}_L^0)$$

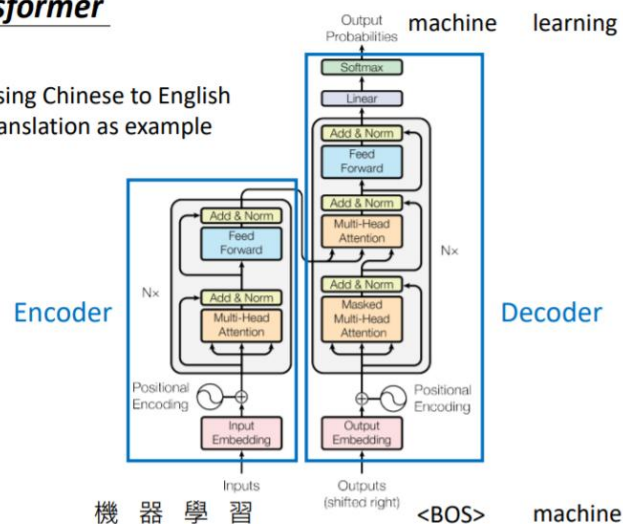
# ViT vs. Transformer

1. ViT only uses the encoder part
2. Split an image into patches and embed them as tokens.  
(Word vectors as tokens in Transformer)
3. Add one additional learnable token: [CLS] token  
(It serves as a conceptual summary about the whole image tokens)
4. Different positional encoding methods



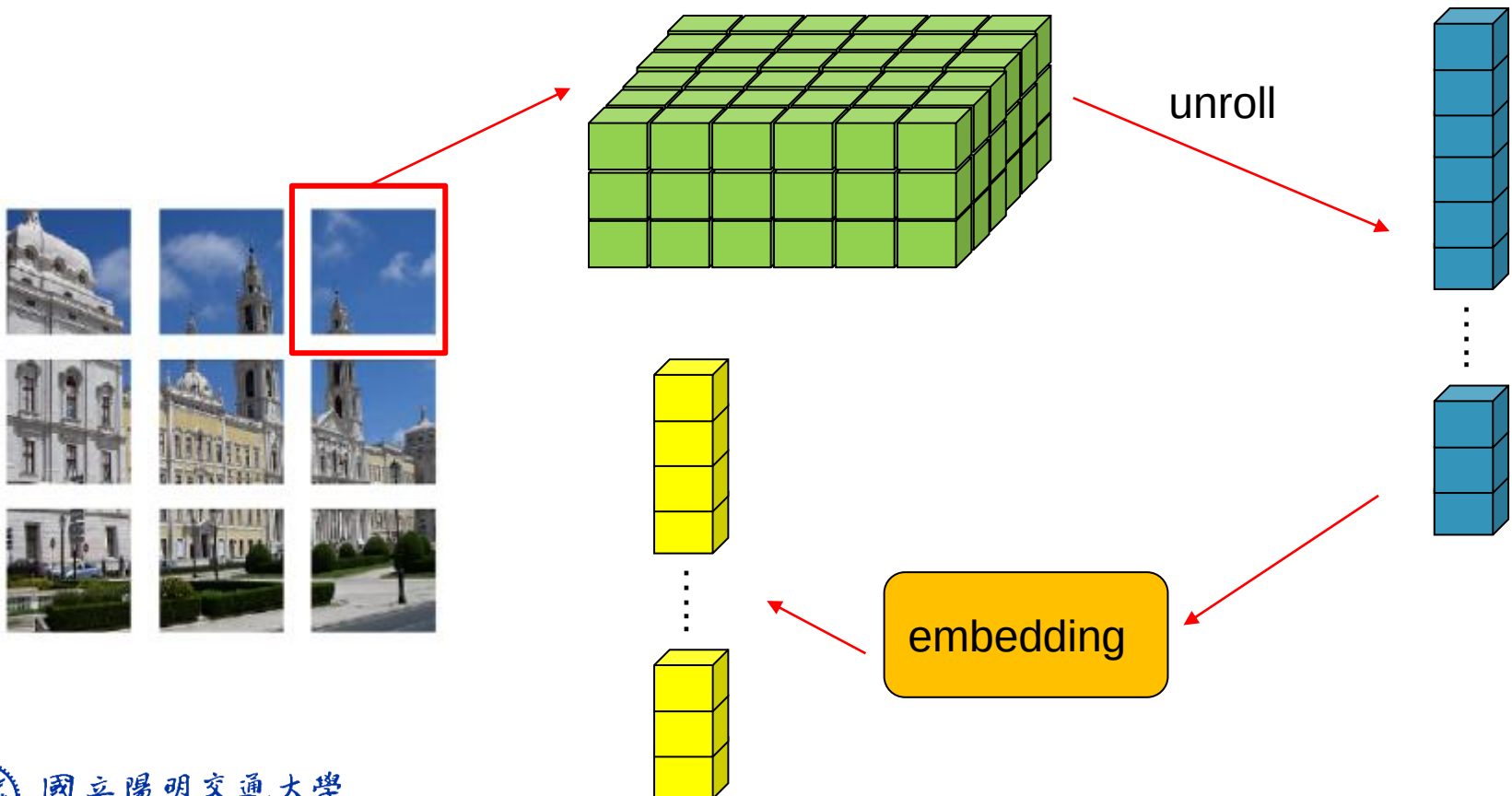
## Transformer

Using Chinese to English translation as example



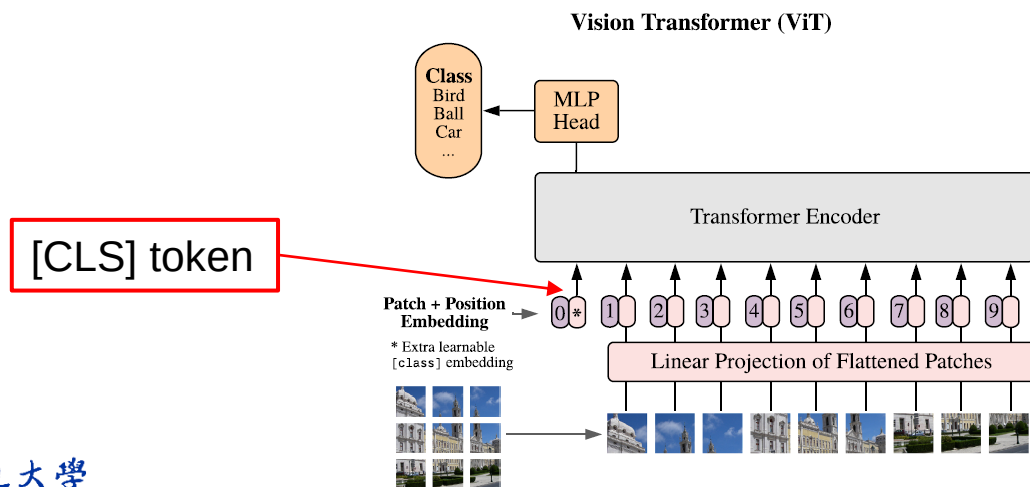
# ViT vs. Transformer

- 2. Split an image into patches and embed them as tokens



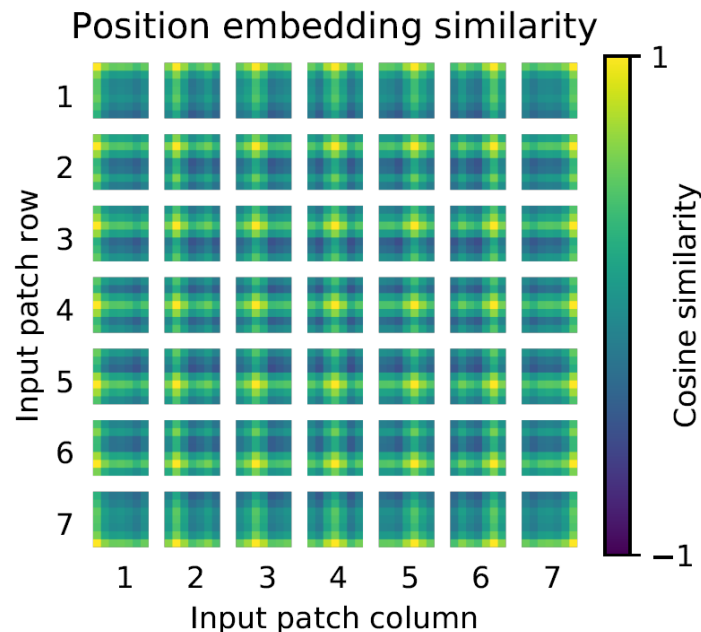
# ViT vs. Transformer

- 3. Add one additional learnable token: [CLS] token
  - Original transformer is designed for NLP (sec2sec task), so its output is also a sequence (a series of word vectors).
  - ViT is used for classification task (only 1 output vector). It would be unfair to choose one particular token for image prediction. Hence, one additional learnable token is included as conceptual summary about the whole image tokens and for final prediction

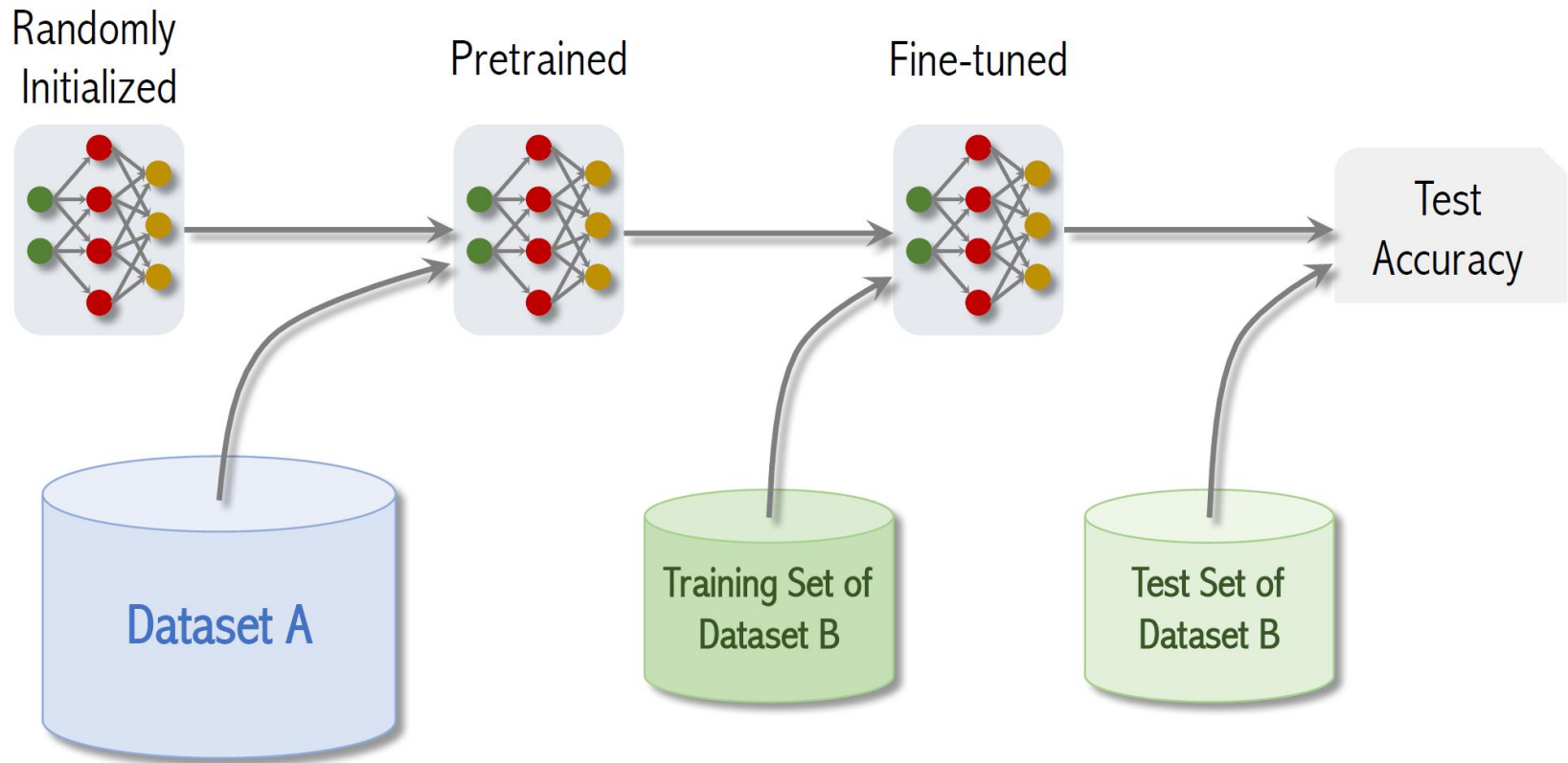


# ViT vs. Transformer

- 4. Different positional encoding methods
  - Positional encoding in the original transformer is given by using sine and cosine functions
  - Positional encoding in ViT is learnable
    - ◆ Cosine similarity among the learned position encoding



# Pre-training, fine-tuning, and testing



# Experiments: Network configurations

- Three variants of ViT of different sizes

| Model     | Layers | Hidden size $D$ | MLP size | Heads | Params |
|-----------|--------|-----------------|----------|-------|--------|
| ViT-Base  | 12     | 768             | 3072     | 12    | 86M    |
| ViT-Large | 24     | 1024            | 4096     | 16    | 307M   |
| ViT-Huge  | 32     | 1280            | 5120     | 16    | 632M   |

- ViT-L/16: the Large variant with 16 x 16 input patch size
- ViT-H/14: the Huge variant with 14 x 14 input patch size





# Competing methods and datasets

- Competing methods
  - BiT (Big Transfer): A variant of ResNet
  - Noisy Student: A variant of EfficientNet
- Datasets for pre-training
  - ImageNet: 1.3M images of 1K classes (small)
  - ImageNet-21K: 14M images of 21K classes (medium)
  - JFT: 300M images of 18K classes (large)



# Experimental results: Comparison with SoTA

- With the model pre-trained on ImageNet-21K, ViT is comparable with (or slightly underperforms) ResNet
- With the model **pre-trained on JFT**, ViT slightly outperforms ResNet, while taking less computational resources for pre-training

|                    | Ours-JFT<br>(ViT-H/14)  | Ours-JFT<br>(ViT-L/16)  | Ours-I21k<br>(ViT-L/16) | BiT-L<br>(ResNet152x4) | Noisy Student<br>(EfficientNet-L2) |
|--------------------|-------------------------|-------------------------|-------------------------|------------------------|------------------------------------|
| ImageNet           | <b>88.55</b> $\pm 0.04$ | 87.76 $\pm 0.03$        | 85.30 $\pm 0.02$        | 87.54 $\pm 0.02$       | 88.4/88.5*                         |
| ImageNet ReaL      | <b>90.72</b> $\pm 0.05$ | 90.54 $\pm 0.03$        | 88.62 $\pm 0.05$        | 90.54                  | 90.55                              |
| CIFAR-10           | <b>99.50</b> $\pm 0.06$ | 99.42 $\pm 0.03$        | 99.15 $\pm 0.03$        | 99.37 $\pm 0.06$       | —                                  |
| CIFAR-100          | <b>94.55</b> $\pm 0.04$ | 93.90 $\pm 0.05$        | 93.25 $\pm 0.05$        | 93.51 $\pm 0.08$       | —                                  |
| Oxford-IIIT Pets   | <b>97.56</b> $\pm 0.03$ | 97.32 $\pm 0.11$        | 94.67 $\pm 0.15$        | 96.62 $\pm 0.23$       | —                                  |
| Oxford Flowers-102 | 99.68 $\pm 0.02$        | <b>99.74</b> $\pm 0.00$ | 99.61 $\pm 0.02$        | 99.63 $\pm 0.03$       | —                                  |
| VTAB (19 tasks)    | <b>77.63</b> $\pm 0.23$ | 76.28 $\pm 0.46$        | 72.72 $\pm 0.21$        | 76.29 $\pm 1.70$       | —                                  |
| TPUv3-core-days    | 2.5k                    | 0.68k                   | 0.23k                   | 9.9k                   | 12.3k                              |



# Attention derived by ViT

- Attention maps are inferred by using [Attention Rollout](#) [Abnar and Zuidema, ACL'20]
- ViT attends to regions semantically relevant to classification



# Outline

- Transformer
- ViT
- DeiT
- Swin Transformer



# DeiT (Data-efficient image Transformers)

[Houvron et al., ICML'21]

- Neural networks purely based on attention, such as ViT, have shown the effectiveness on image classification
- Such high-performing transformers are pre-trained with hundreds of millions of images, such as those from JFT
  - The paper of ViT states “ViT does not generalize well when trained on insufficient amounts of data.”
- The JFT dataset is unavailable in public, so most researchers cannot reproduce the result
- DeiT aims to train a high-performing transformer
  - Of a small model size (86M parameters)
  - On a smaller dataset, e.g., ImageNet1K (1.4 millions images)
  - In a single computer (8-GPU node) less than 3 days



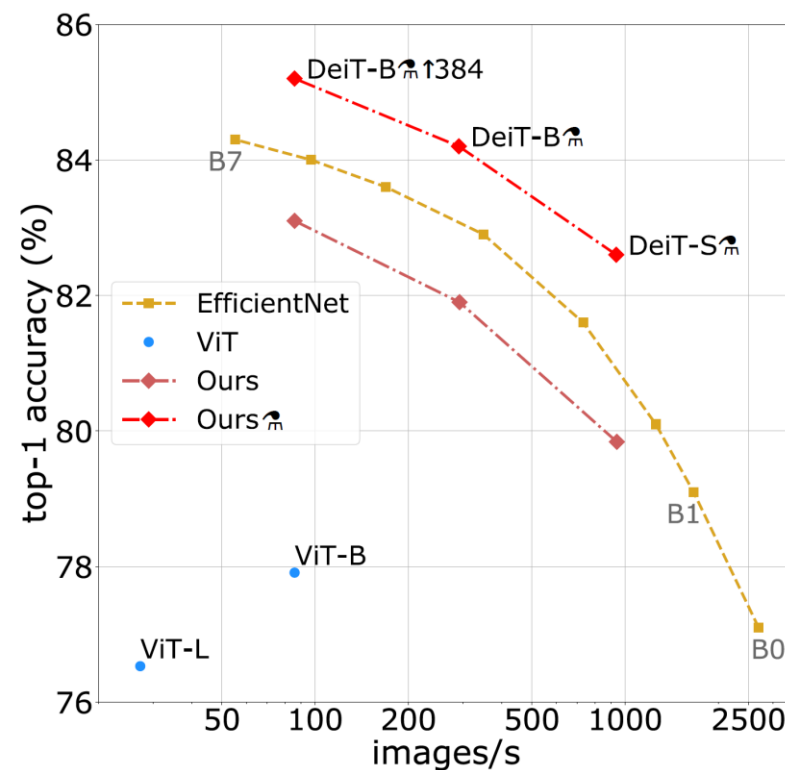
# DeiT

- It adopts the model architecture of ViT
- In order to train on a smaller dataset, this work conducts the detailed research on
  - Data efficient training:
    - ◆ Optimization solver
    - ◆ Data augmentation
    - ◆ Regularization
  - Apply **distillation** to ViT



# Comparison among DeiT, ViT, and EfficientNet

- Throughput vs. accuracy
  - ImageNet1k only
  - Images per second on V100
- ViT is not effective on a small dataset
- Ours (DeiT) uses a data-starving regime
- With **transformer-specific distillation**, DeiT<sub>m</sub> outperforms EfficientNet



# Data-efficient training

- Optimization solver
  - AdamW [Loshchilov & Hutter, arXiv'17], Adam with weight decay
- Data augmentation
  - Rand-Augment [Cubuk et al. arXiv'19]
  - Mixup [Zhang et al. arXiv'17]
  - CutMix [Yun et al. arXiv'19]
- Regularization
  - Stochastic depth [Huang et al. ECCV'16]
  - Random erasing [Zhong et al. AAAI'20]
  - Repeated augmentation [Hoffer et al. CVPR'20]





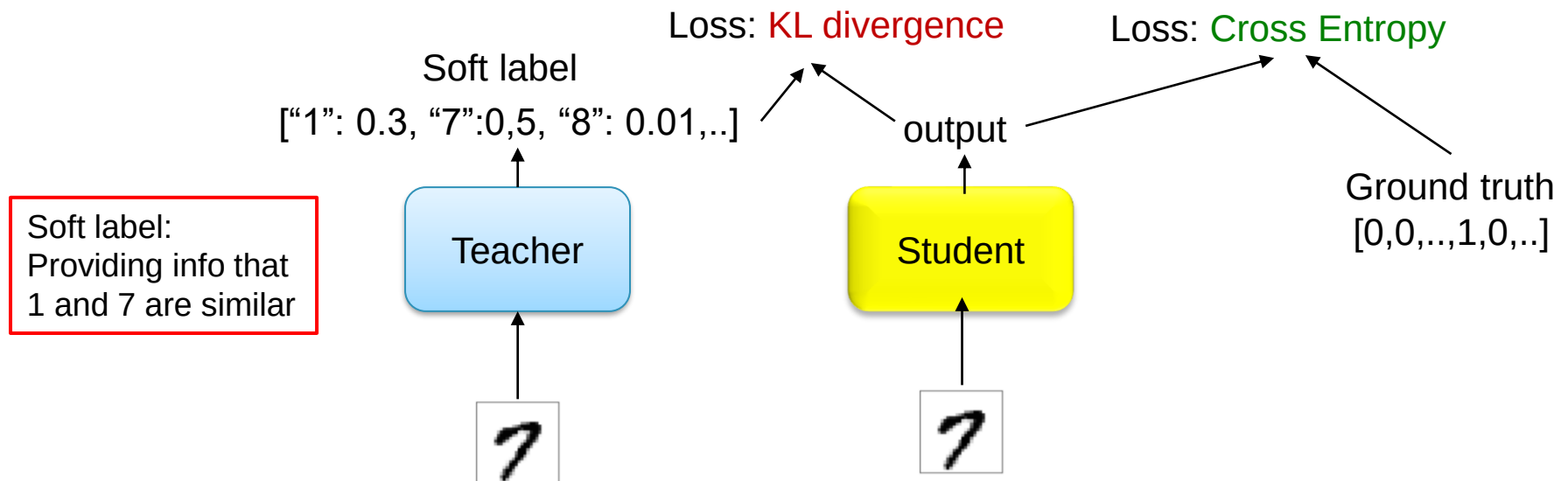
# Ablation studies of these components

| Ablation on ↓     | Pre-training | Fine-tuning | Rand-Augment | AutoAug | Mixup | CutMix | Erasing | Stoch. Depth | Repeated Aug. | Dropout | Exp. Moving Avg. | top-1 accuracy               |                             |
|-------------------|--------------|-------------|--------------|---------|-------|--------|---------|--------------|---------------|---------|------------------|------------------------------|-----------------------------|
|                   |              |             |              |         |       |        |         |              |               |         |                  | pre-trained 224 <sup>2</sup> | fine-tuned 384 <sup>2</sup> |
| none: DeiT-B      | adamw        | adamw       | ✓            | ✗       | ✓     | ✓      | ✓       | ✓            | ✓             | ✗       | ✗                | 81.8 ±0.2                    | 83.1 ±0.1                   |
| optimizer         | SGD          | adamw       | ✓            | ✗       | ✓     | ✓      | ✓       | ✓            | ✓             | ✗       | ✗                | 74.5                         | 77.3                        |
|                   | adamw        | SGD         | ✓            | ✗       | ✓     | ✓      | ✓       | ✓            | ✓             | ✗       | ✗                | 81.8                         | 83.1                        |
| data augmentation | adamw        | adamw       | ✗            | ✗       | ✓     | ✓      | ✓       | ✓            | ✓             | ✗       | ✗                | 79.6                         | 80.4                        |
|                   | adamw        | adamw       | ✗            | ✓       | ✓     | ✓      | ✓       | ✓            | ✓             | ✗       | ✗                | 81.2                         | 81.9                        |
|                   | adamw        | adamw       | ✓            | ✗       | ✗     | ✓      | ✓       | ✓            | ✓             | ✗       | ✗                | 78.7                         | 79.8                        |
|                   | adamw        | adamw       | ✓            | ✗       | ✓     | ✗      | ✓       | ✓            | ✓             | ✗       | ✗                | 80.0                         | 80.6                        |
|                   | adamw        | adamw       | ✓            | ✗       | ✗     | ✗      | ✓       | ✓            | ✓             | ✗       | ✗                | 75.8                         | 76.7                        |
| regularization    | adamw        | adamw       | ✓            | ✗       | ✓     | ✓      | ✗       | ✓            | ✓             | ✗       | ✗                | 4.3*                         | 0.1                         |
|                   | adamw        | adamw       | ✓            | ✗       | ✓     | ✓      | ✓       | ✗            | ✓             | ✗       | ✗                | 3.4*                         | 0.1                         |
|                   | adamw        | adamw       | ✓            | ✗       | ✓     | ✓      | ✓       | ✓            | ✗             | ✗       | ✗                | 76.5                         | 77.4                        |
|                   | adamw        | adamw       | ✓            | ✗       | ✓     | ✓      | ✓       | ✓            | ✓             | ✓       | ✗                | 81.3                         | 83.1                        |
|                   | adamw        | adamw       | ✓            | ✗       | ✓     | ✓      | ✓       | ✓            | ✓             | ✗       | ✓                | 81.9                         | 83.1                        |



# Distillation (蒸餾)

- Distillation
  - There are a **teacher network** and a **student network**
  - The student network is derived from two sources
    - ◆ **Ground-truth labels** via **cross entropy loss**
    - ◆ **Soft-label outputs** by the teacher network via **Kullback-Leibler (KL) divergence**



# Soft distillation and hard distillation

- Two types of distillation:

- Soft distillation:

- ◆ The teacher's output is soft labels and the used loss is KL divergence

$$\mathcal{L}_{\text{global}} = (1 - \lambda) \mathcal{L}_{\text{CE}}(\overset{\text{Softmax}}{\psi(Z_s)}, \overset{\text{Ground truth}}{y}) + \lambda \tau^2 \text{KL}(\psi(Z_s/\tau), \psi(\overset{\text{Teacher output (soft)}}{Z_t}/\tau)).$$

Student output

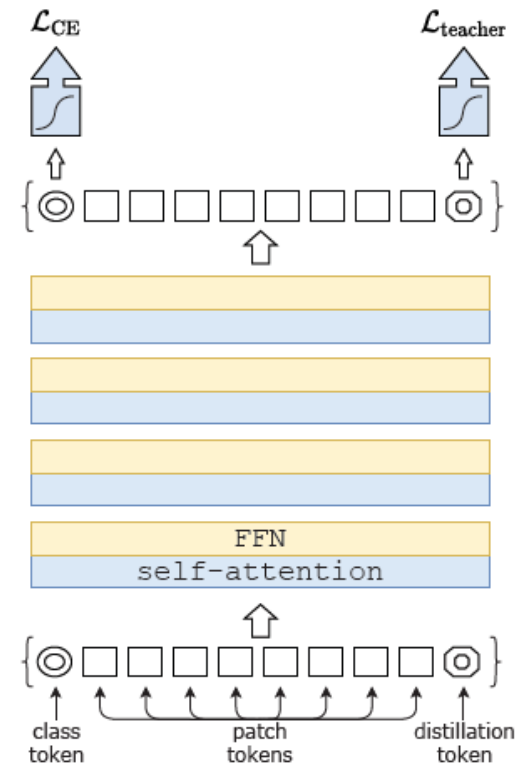
- Hard distillation:

- ◆ The teacher's output is hard labels and the used loss is cross entropy

$$\mathcal{L}_{\text{global}}^{\text{hardDistill}} = \frac{1}{2} \mathcal{L}_{\text{CE}}(\psi(Z_s), y) + \frac{1}{2} \mathcal{L}_{\text{CE}}(\psi(Z_s), \overset{\text{Teacher output (hard)}}{y_t}).$$

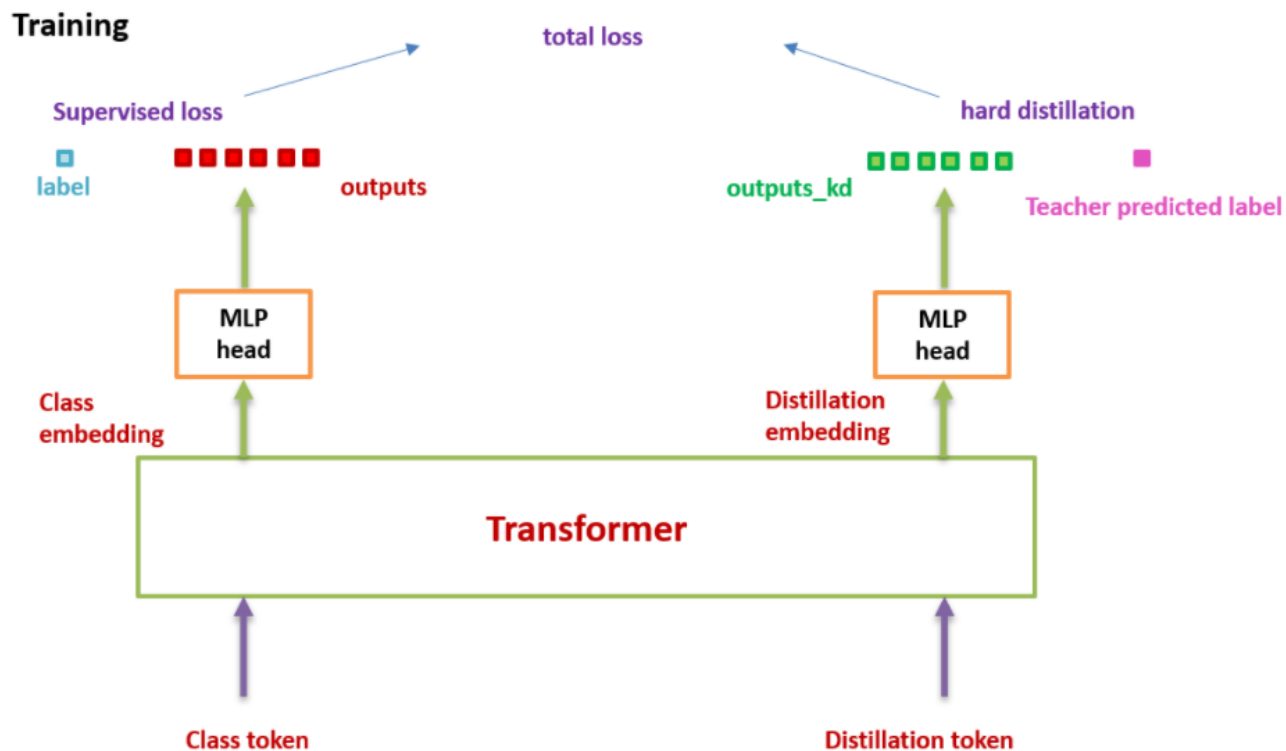
# Transformer-specific distillation

- Distillation on transformer
  - Teacher network: RegNetY (CNN-based model, a variant of ResNet)
  - Additional token: “distillation token”
    - ◆ [cls] token: Aim to get close to GT label
    - ◆ [distill] token: Aim to get close to the teacher network’s output
    - ◆ Patch tokens



# Illustration

- Distillation on transformer



(Source: <https://zhuanlan.zhihu.com/p/349315675>)

# Why choosing CNN as the teacher model

- CNN can be learned with a smaller dataset (compared with Transformer)
- CNN has inductive bias about images, such as local connectivity and weight sharing
- The inductive bias can be inherited by the transformers through distillation



# Experiment: Network architecture

- DeiT models of three different sizes

| Model   | embedding dimension | #heads | #layers | #params | training resolution | throughput (im/sec) |
|---------|---------------------|--------|---------|---------|---------------------|---------------------|
| DeiT-Ti | 192                 | 3      | 12      | 5M      | 224                 | 2536                |
| DeiT-S  | 384                 | 6      | 12      | 22M     | 224                 | 940                 |
| DeiT-B  | 768                 | 12     | 12      | 86M     | 224                 | 292                 |

- DeiT-B has the same architecture as ViT-B, but is derived by further considering 1) data-efficient training and 2) distillation
- DeiT-S and DeiT-Ti are two smaller variants of DeiT-B by reducing the dimension of embedding and the number of heads



# Experiment: Distillation from a CNN or a transformer

- Teacher model
  - CNN: one of the four variants of RegNet
  - Transformer: DeiT-B

| Teacher      |      | Student: DeiT-B |      |
|--------------|------|-----------------|------|
| Models       | acc. | pretrain        | ↑384 |
| DeiT-B       | 81.8 | 81.9            | 83.1 |
| RegNetY-4GF  | 80.0 | 82.7            | 83.6 |
| RegNetY-8GF  | 81.7 | 82.7            | 83.8 |
| RegNetY-12GF | 82.4 | 83.0            | 83.9 |
| RegNetY-16GF | 82.9 | 83.0            | 84.0 |

- Using a convnet teacher gives better performance





# Experiment: Transformer-specific distillation

- Results

| method ↓                               | Supervision |         | ImageNet top-1 (%) |       |       |       |
|----------------------------------------|-------------|---------|--------------------|-------|-------|-------|
|                                        | label       | teacher | Ti 224             | S 224 | B 224 | B↑384 |
| DeiT– no distillation                  | ✓           | ✗       | 72.2               | 79.8  | 81.8  | 83.1  |
| DeiT– usual distillation               | ✗           | soft    | 72.2               | 79.8  | 81.8  | 83.2  |
| DeiT– hard distillation                | ✗           | hard    | 74.3               | 80.9  | 83.0  | 84.0  |
| DeiT <sub>m</sub> : class embedding    | ✓           | hard    | 73.9               | 80.9  | 83.0  | 84.2  |
| DeiT <sub>m</sub> : distil. embedding  | ✓           | hard    | 74.6               | 81.1  | 83.1  | 84.4  |
| DeiT <sub>m</sub> : class+distillation | ✓           | hard    | 74.5               | 81.2  | 83.4  | 84.5  |

- Hard distillation is better than soft distillation
- Transformer-specific distillation works better standard distillation
  - The extra distillation token in DeiT<sub>m</sub> helps
- Class embedding and distillation embedding are useful for classification and complementary



# Comparison with SoTAs

- Train on ImageNet1K
- DeiT-B outperforms ViT-B
  - Data-efficient training
  - Distillation
- Fine-tuning with high-resolution images helps
- More training epochs improve performance
- Comparable with SoTAs

| Network                   | #param. | image throughput<br>size (image/s) |        | ImNet<br>top-1 |
|---------------------------|---------|------------------------------------|--------|----------------|
| Convnets                  |         |                                    |        |                |
| ResNet-18 [21]            | 12M     | 224 <sup>2</sup>                   | 4458.4 | 69.8           |
| ResNet-50 [21]            | 25M     | 224 <sup>2</sup>                   | 1226.1 | 76.2           |
| ResNet-101 [21]           | 45M     | 224 <sup>2</sup>                   | 753.6  | 77.4           |
| ResNet-152 [21]           | 60M     | 224 <sup>2</sup>                   | 526.4  | 78.3           |
| RegNetY-4GF [40]★         | 21M     | 224 <sup>2</sup>                   | 1156.7 | 80.0           |
| RegNetY-8GF [40]★         | 39M     | 224 <sup>2</sup>                   | 591.6  | 81.7           |
| RegNetY-16GF [40]★        | 84M     | 224 <sup>2</sup>                   | 334.7  | 82.9           |
| EfficientNet-B0 [48]      | 5M      | 224 <sup>2</sup>                   | 2694.3 | 77.1           |
| EfficientNet-B1 [48]      | 8M      | 240 <sup>2</sup>                   | 1662.5 | 79.1           |
| EfficientNet-B2 [48]      | 9M      | 260 <sup>2</sup>                   | 1255.7 | 80.1           |
| EfficientNet-B3 [48]      | 12M     | 300 <sup>2</sup>                   | 732.1  | 81.6           |
| EfficientNet-B4 [48]      | 19M     | 380 <sup>2</sup>                   | 349.4  | 82.9           |
| EfficientNet-B5 [48]      | 30M     | 456 <sup>2</sup>                   | 169.1  | 83.6           |
| EfficientNet-B6 [48]      | 43M     | 528 <sup>2</sup>                   | 96.9   | 84.0           |
| EfficientNet-B7 [48]      | 66M     | 600 <sup>2</sup>                   | 55.1   | 84.3           |
| EfficientNet-B5 RA [12]   | 30M     | 456 <sup>2</sup>                   | 96.9   | 83.7           |
| EfficientNet-B7 RA [12]   | 66M     | 600 <sup>2</sup>                   | 55.1   | 84.7           |
| KDforAA-B8                | 87M     | 800 <sup>2</sup>                   | 25.2   | 85.8           |
| Transformers              |         |                                    |        |                |
| ViT-B/16 [15]             | 86M     | 384 <sup>2</sup>                   | 85.9   | 77.9           |
| ViT-L/16 [15]             | 307M    | 384 <sup>2</sup>                   | 27.3   | 76.5           |
| DeiT-Ti                   | 5M      | 224 <sup>2</sup>                   | 2536.5 | 72.2           |
| DeiT-S                    | 22M     | 224 <sup>2</sup>                   | 940.4  | 79.8           |
| DeiT-B                    | 86M     | 224 <sup>2</sup>                   | 292.3  | 81.8           |
| DeiT-B↑384                | 86M     | 384 <sup>2</sup>                   | 85.9   | 83.1           |
| DeiT-Ti🐼                  | 6M      | 224 <sup>2</sup>                   | 2529.5 | 74.5           |
| DeiT-S🐼                   | 22M     | 224 <sup>2</sup>                   | 936.2  | 81.2           |
| DeiT-B🐼                   | 87M     | 224 <sup>2</sup>                   | 290.9  | 83.4           |
| DeiT-Ti🐼 / 1000 epochs    | 6M      | 224 <sup>2</sup>                   | 2529.5 | 76.6           |
| DeiT-S🐼 / 1000 epochs     | 22M     | 224 <sup>2</sup>                   | 936.2  | 82.6           |
| DeiT-B🐼 / 1000 epochs     | 87M     | 224 <sup>2</sup>                   | 290.9  | 84.2           |
| DeiT-B🐼↑384               | 87M     | 384 <sup>2</sup>                   | 85.8   | 84.5           |
| DeiT-B🐼↑384 / 1000 epochs | 87M     | 384 <sup>2</sup>                   | 85.8   | 85.2           |



# Outline

- Transformer
- ViT
- DeiT
- Swin Transformer



# Introduction

[Liu et al., ICCV'21]

**Best Paper Award!**

- **Swin** Transformer: Hierarchical Vision Transformer using Shifted **Windows**
  - Hierarchical: Multi-scale learning typically helps
  - Shifted windows: Reduced complexity for self-attention
- A general backbone for several computer vision applications
  - Extract features using self-attention
  - Promising results on image classification, object detection, and semantic segmentation
- **Linear** computational complexity to image size
  - The complexity grows quadratically to image size in most transformers



# Motivation

- Challenges of transformers from NLP to CV
  - Scale:
    - ◆ Word tokens are basic elements in language transformers
    - ◆ Visual tokens (pixels or patches) can vary a lot in scale
  - Memory usage and computation cost:
    - ◆ Both storage and computational complexities grow quadratically with respect to the number of tokens



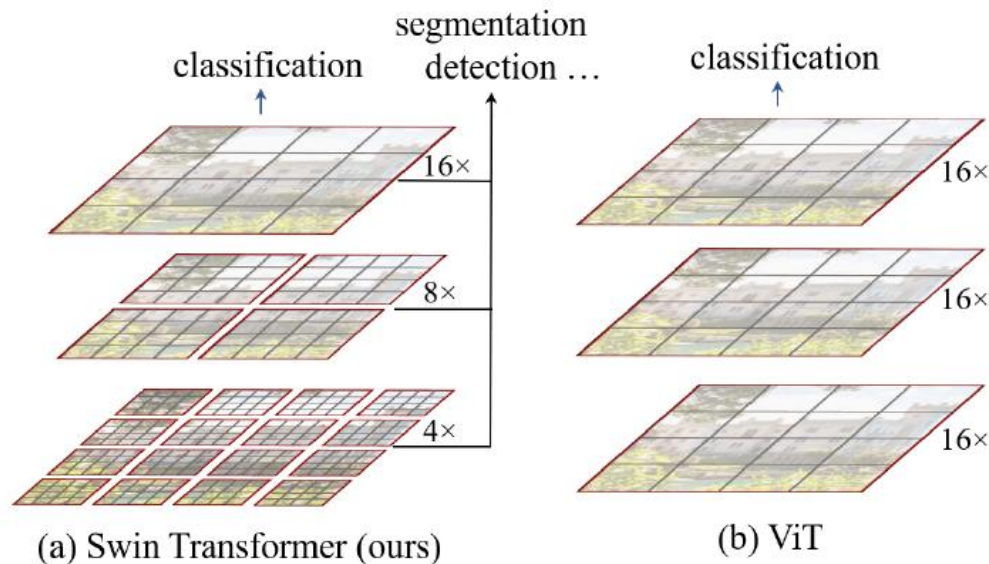
# Properties of Swin Transformer

- Hierarchical feature maps by merging image patches
- Linear computational complexity to image size by computing self-attention window-by-window

$$\Omega(\text{MSA}) = 4hwC^2 + 2(hw)^2C,$$

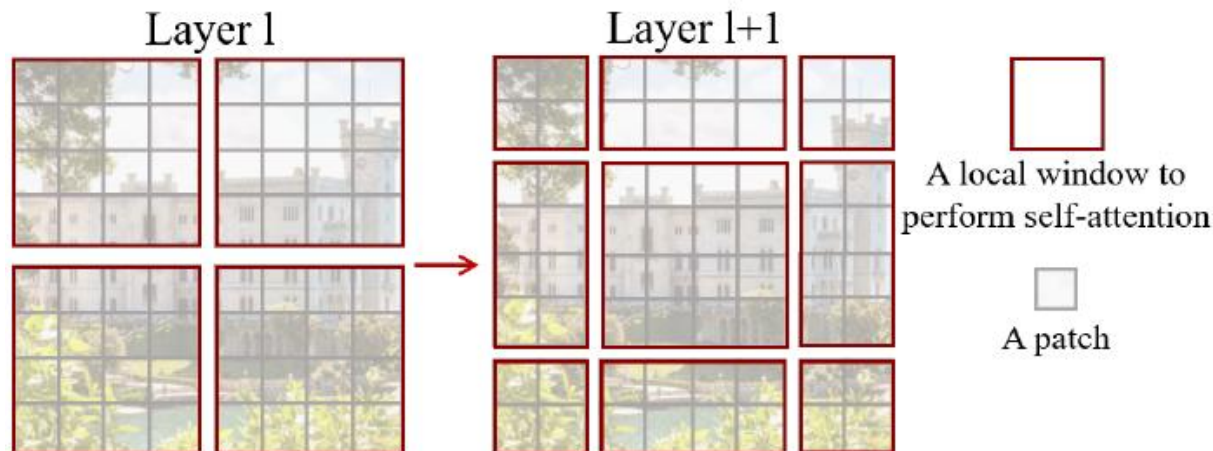
$$\Omega(\text{W-MSA}) = 4hwC^2 + 2M^2hwC,$$

- General-purpose backbone



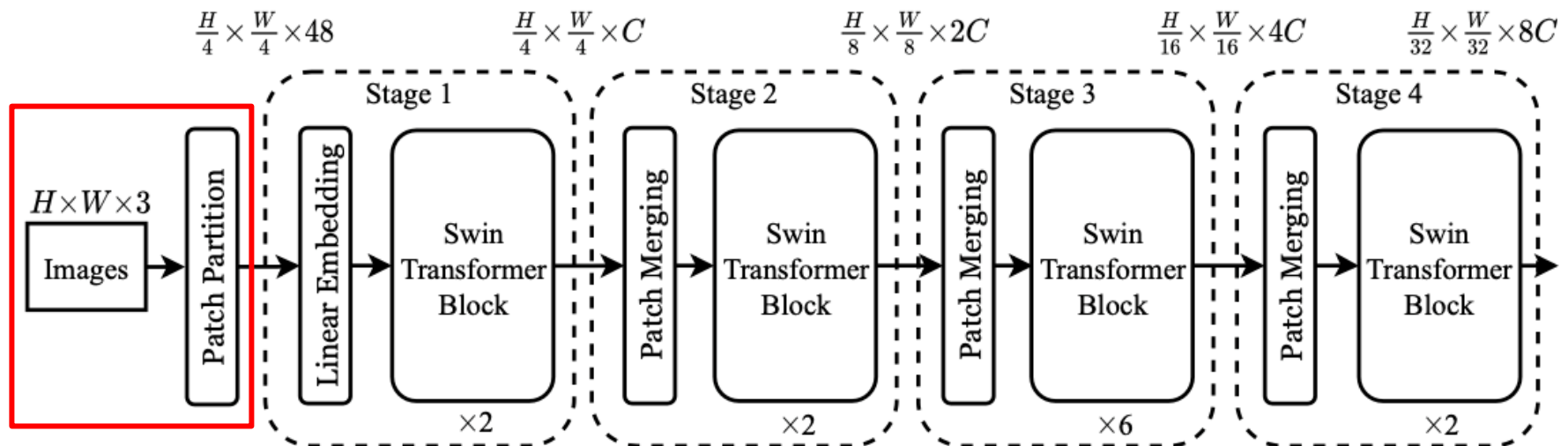
# Shifted window

- Window-based attention suffers from block effect
- Shifted window is implemented for **two successive layers**
- In layer  $l$ , a regular window partitioning scheme is adopted
- In layer  $l+1$ , the window partitioning is shifted



# Network architecture: Input

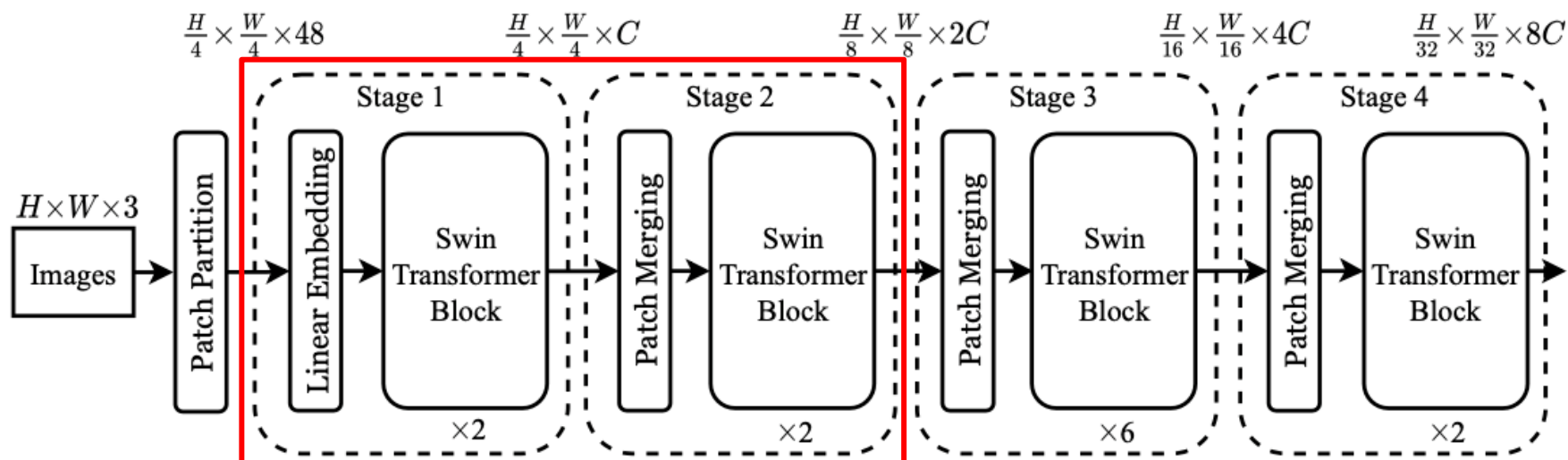
- Given a color image of resolution  $H \times W$  as input
- We partition it into patches, each of which is of size  $4 \times 4$ 
  - The number of tokens is  $H/4 \times W/4$
  - The feature dimension of each token is 48 ( $= 4 \times 4 \times 3$ )
- This architecture is composed of 4 stages, one for each scale





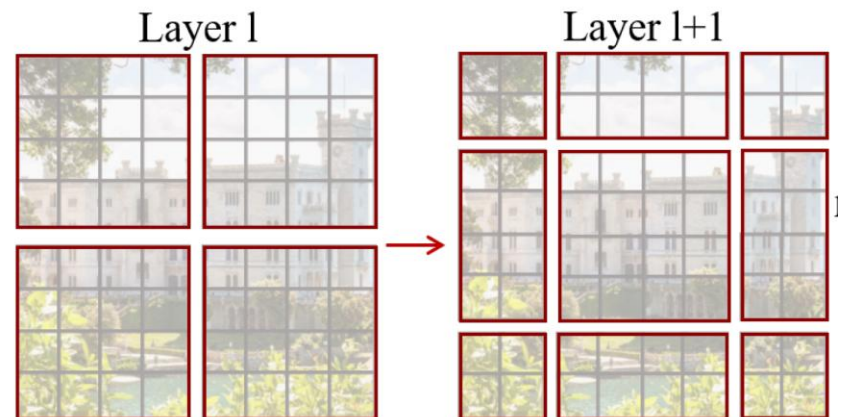
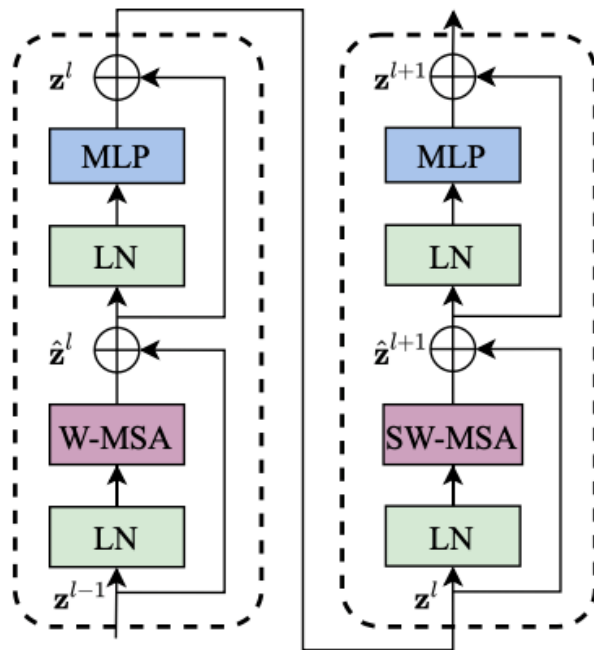
# Network architecture: From stage 1 to stage 2

- Linear embedding: Compute the features of each token
- Swin Transformer Block: Each stage has an even number of blocks where shifted window mechanism is used



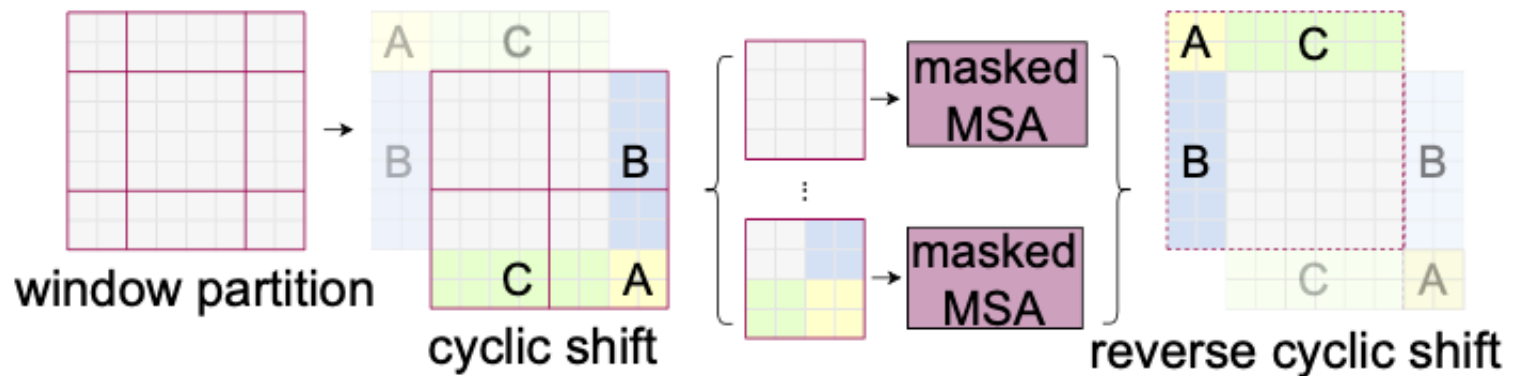
# Two successive Swin Transformer blocks

- W-MSA: Multi-head self-attention with windowing
- SW-MSA: Multi-head self-attention with shifted windowing
- First block uses a regular window partitioning
- Next block adopts a shifted window configuration



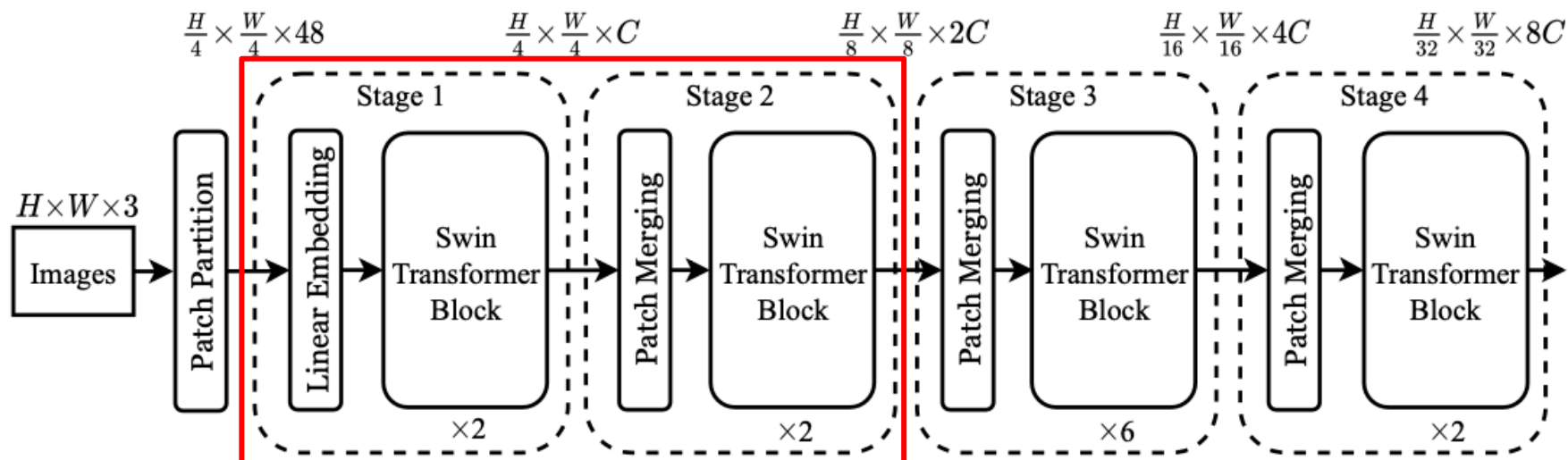
# Cyclic shift

- After shifted windowing, some of windows are smaller than  $M \times M$  (window size)
- For parallel computing, we prefer to have windows of the same sizes
- Use cyclic shift instead of padding the smaller windows, because it is more efficient



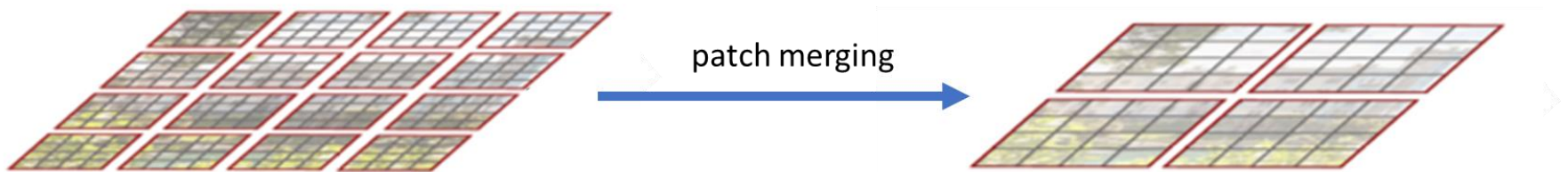
# Network architecture: From stage 1 to stage 2

- Linear embedding: Compute the features of each token
- Swin Transformer Block: Each stage has an even number of blocks where shifted window mechanism is used
- Patch merging layer to reduce the number of patches for multi-scale learning



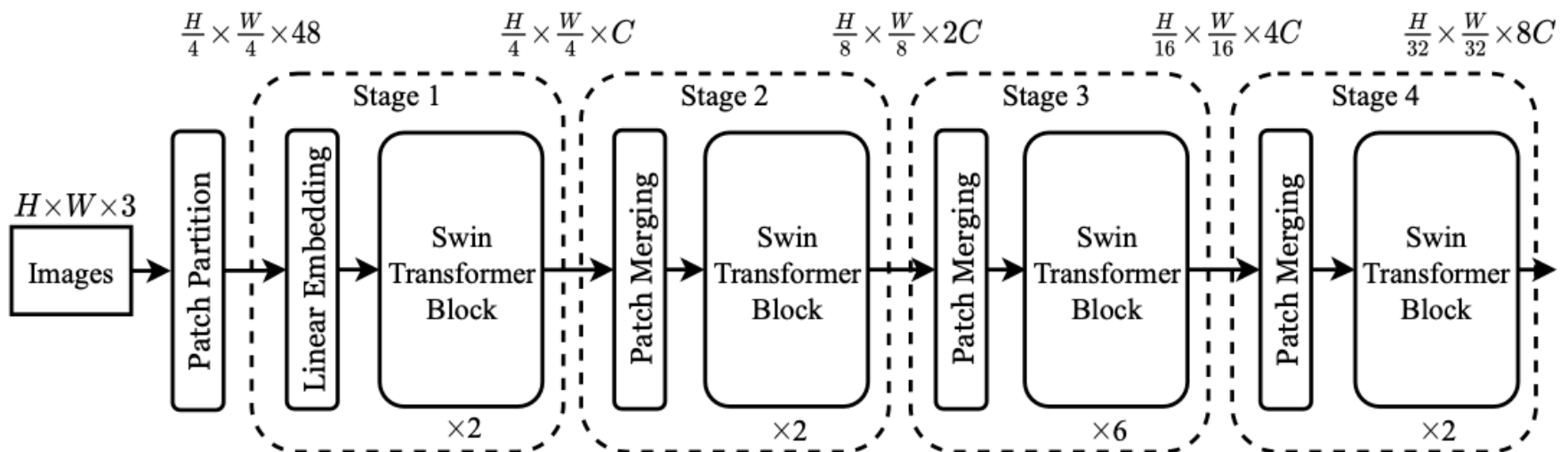
# Patch merging layer

- It concatenates the features of each group of  $2 \times 2$  neighboring patches
- The dimension of a merged token is  $4C$
- A linear layer is applied to reduce the dimension from  $4C$  to  $2C$



# Experiments: Model sizes

- Four Swin Transformer models of different sizes
  - Swin-T:  $C = 96$ , layer numbers =  $\{2, 2, 6, 2\}$
  - Swin-S:  $C = 96$ , layer numbers =  $\{2, 2, 18, 2\}$
  - Swin-B:  $C = 128$ , layer numbers =  $\{2, 2, 18, 2\}$
  - Swin-L:  $C = 192$ , layer numbers =  $\{2, 2, 18, 2\}$



# Experiments: Object recognition on ImageNet

- Comparison among RegNetY, EfficientNet, ViT, DeiT, and Swin-T

(a) Regular ImageNet-1K trained models

| method           | image size       | #param. | FLOPs  | throughput (image / s) | ImageNet top-1 acc. |
|------------------|------------------|---------|--------|------------------------|---------------------|
| RegNetY-4G [48]  | 224 <sup>2</sup> | 21M     | 4.0G   | 1156.7                 | 80.0                |
| RegNetY-8G [48]  | 224 <sup>2</sup> | 39M     | 8.0G   | 591.6                  | 81.7                |
| RegNetY-16G [48] | 224 <sup>2</sup> | 84M     | 16.0G  | 334.7                  | 82.9                |
| EffNet-B3 [58]   | 300 <sup>2</sup> | 12M     | 1.8G   | 732.1                  | 81.6                |
| EffNet-B4 [58]   | 380 <sup>2</sup> | 19M     | 4.2G   | 349.4                  | 82.9                |
| EffNet-B5 [58]   | 456 <sup>2</sup> | 30M     | 9.9G   | 169.1                  | 83.6                |
| EffNet-B6 [58]   | 528 <sup>2</sup> | 43M     | 19.0G  | 96.9                   | 84.0                |
| EffNet-B7 [58]   | 600 <sup>2</sup> | 66M     | 37.0G  | 55.1                   | 84.3                |
| ViT-B/16 [20]    | 384 <sup>2</sup> | 86M     | 55.4G  | 85.9                   | 77.9                |
| ViT-L/16 [20]    | 384 <sup>2</sup> | 307M    | 190.7G | 27.3                   | 76.5                |
| DeiT-S [63]      | 224 <sup>2</sup> | 22M     | 4.6G   | 940.4                  | 79.8                |
| DeiT-B [63]      | 224 <sup>2</sup> | 86M     | 17.5G  | 292.3                  | 81.8                |
| DeiT-B [63]      | 384 <sup>2</sup> | 86M     | 55.4G  | 85.9                   | 83.1                |
| Swin-T           | 224 <sup>2</sup> | 29M     | 4.5G   | 755.2                  | 81.3                |
| Swin-S           | 224 <sup>2</sup> | 50M     | 8.7G   | 436.9                  | 83.0                |
| Swin-B           | 224 <sup>2</sup> | 88M     | 15.4G  | 278.1                  | 83.5                |
| Swin-B           | 384 <sup>2</sup> | 88M     | 47.0G  | 84.7                   | 84.5                |

# Experiments: Object recognition on ImageNet

- Comparison among ResNet, ViT, and Swin-T

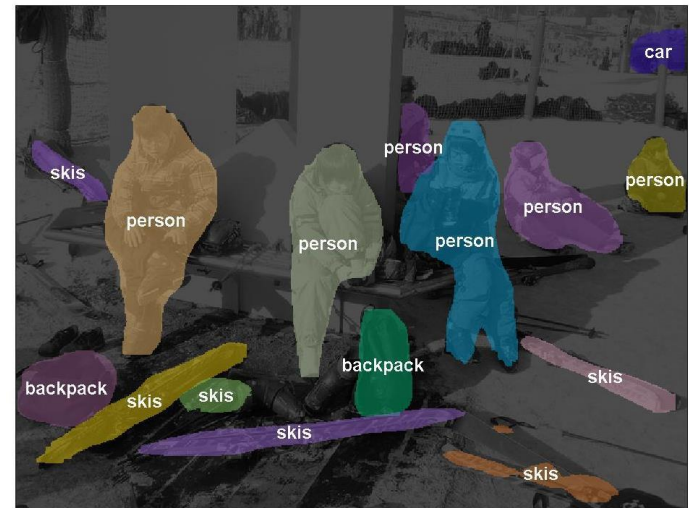
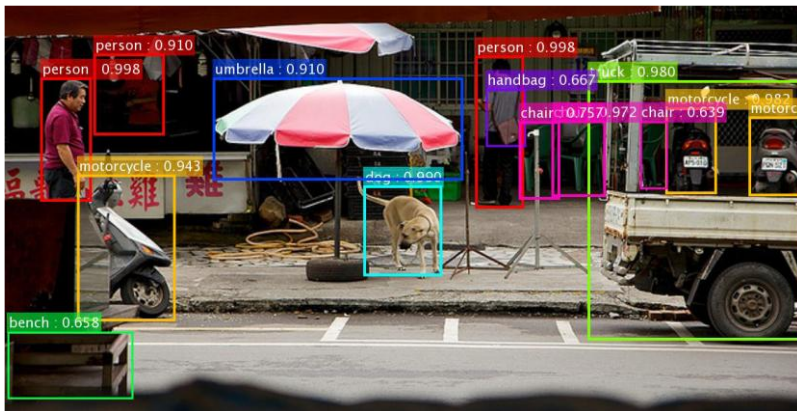
**(b) ImageNet-22K pre-trained models**

| method        | image size       | #param. | FLOPs  | throughput (image / s) | ImageNet top-1 acc. |
|---------------|------------------|---------|--------|------------------------|---------------------|
| R-101x3 [38]  | 384 <sup>2</sup> | 388M    | 204.6G | -                      | 84.4                |
| R-152x4 [38]  | 480 <sup>2</sup> | 937M    | 840.5G | -                      | 85.4                |
| ViT-B/16 [20] | 384 <sup>2</sup> | 86M     | 55.4G  | 85.9                   | 84.0                |
| ViT-L/16 [20] | 384 <sup>2</sup> | 307M    | 190.7G | 27.3                   | 85.2                |
| Swin-B        | 224 <sup>2</sup> | 88M     | 15.4G  | 278.1                  | 85.2                |
| Swin-B        | 384 <sup>2</sup> | 88M     | 47.0G  | 84.7                   | 86.4                |
| Swin-L        | 384 <sup>2</sup> | 197M    | 103.9G | 42.1                   | 87.3                |



# Object detection and instance segmentation

- Evaluate metrics:  $AP^{bb}$  (box average precision) and  $AP^{mask}$  (mask average precision)



# Experiments: Object detection on MS COCO dataset

- R-50 vs. Swin-T on detection with different detection frameworks

(a) Various frameworks

| Method      | Backbone | AP <sup>box</sup> | AP <sub>50</sub> <sup>box</sup> | AP <sub>75</sub> <sup>box</sup> | #param. | FLOPs | FPS  |
|-------------|----------|-------------------|---------------------------------|---------------------------------|---------|-------|------|
| Cascade     | R-50     | 46.3              | 64.3                            | 50.5                            | 82M     | 739G  | 18.0 |
| Mask R-CNN  | Swin-T   | <b>50.5</b>       | <b>69.3</b>                     | <b>54.9</b>                     | 86M     | 745G  | 15.3 |
| ATSS        | R-50     | 43.5              | 61.9                            | 47.0                            | 32M     | 205G  | 28.3 |
|             | Swin-T   | <b>47.2</b>       | <b>66.5</b>                     | <b>51.3</b>                     | 36M     | 215G  | 22.3 |
| RepPointsV2 | R-50     | 46.5              | 64.6                            | 50.3                            | 42M     | 274G  | 13.6 |
|             | Swin-T   | <b>50.0</b>       | <b>68.5</b>                     | <b>54.2</b>                     | 45M     | 283G  | 12.0 |
| Sparse      | R-50     | 44.5              | 63.4                            | 48.2                            | 106M    | 166G  | 21.0 |
| R-CNN       | Swin-T   | <b>47.9</b>       | <b>67.3</b>                     | <b>52.3</b>                     | 110M    | 172G  | 18.4 |



# Experiments: Object detection and instance segmentation on MS COCO dataset

- Comparison of different backbones on object detection instance segmentation

(b) Various backbones w. Cascade Mask R-CNN

|                     | AP <sup>box</sup> | AP <sub>50</sub> <sup>box</sup> | AP <sub>75</sub> <sup>box</sup> | AP <sup>mask</sup> | AP <sub>50</sub> <sup>mask</sup> | AP <sub>75</sub> <sup>mask</sup> | param | FLOPs | FPS  |
|---------------------|-------------------|---------------------------------|---------------------------------|--------------------|----------------------------------|----------------------------------|-------|-------|------|
| DeiT-S <sup>†</sup> | 48.0              | 67.2                            | 51.7                            | 41.4               | 64.2                             | 44.3                             | 80M   | 889G  | 10.4 |
| R50                 | 46.3              | 64.3                            | 50.5                            | 40.1               | 61.7                             | 43.4                             | 82M   | 739G  | 18.0 |
| Swin-T              | <b>50.5</b>       | <b>69.3</b>                     | <b>54.9</b>                     | <b>43.7</b>        | <b>66.6</b>                      | <b>47.1</b>                      | 86M   | 745G  | 15.3 |
| X101-32             | 48.1              | 66.5                            | 52.4                            | 41.6               | 63.9                             | 45.2                             | 101M  | 819G  | 12.8 |
| Swin-S              | <b>51.8</b>       | <b>70.4</b>                     | <b>56.3</b>                     | <b>44.7</b>        | <b>67.9</b>                      | <b>48.5</b>                      | 107M  | 838G  | 12.0 |
| X101-64             | 48.3              | 66.4                            | 52.3                            | 41.7               | 64.0                             | 45.1                             | 140M  | 972G  | 10.4 |
| Swin-B              | <b>51.9</b>       | <b>70.9</b>                     | <b>56.5</b>                     | <b>45.0</b>        | <b>68.4</b>                      | <b>48.7</b>                      | 145M  | 982G  | 11.6 |

# Experiments: Semantic segmentation on ADE20K

- Comparison among different backbones on different segmentation frameworks

| ADE20K        |                      | val         | test        | #param. | FLOPs | FPS  |
|---------------|----------------------|-------------|-------------|---------|-------|------|
| Method        | Backbone             | mIoU        | score       |         |       |      |
| DANet [23]    | ResNet-101           | 45.2        | -           | 69M     | 1119G | 15.2 |
| DLab.v3+ [11] | ResNet-101           | 44.1        | -           | 63M     | 1021G | 16.0 |
| ACNet [24]    | ResNet-101           | 45.9        | 38.5        | -       |       |      |
| DNL [71]      | ResNet-101           | 46.0        | 56.2        | 69M     | 1249G | 14.8 |
| OCRNet [73]   | ResNet-101           | 45.3        | 56.0        | 56M     | 923G  | 19.3 |
| UperNet [69]  | ResNet-101           | 44.9        | -           | 86M     | 1029G | 20.1 |
| OCRNet [73]   | HRNet-w48            | 45.7        | -           | 71M     | 664G  | 12.5 |
| DLab.v3+ [11] | ResNeSt-101          | 46.9        | 55.1        | 66M     | 1051G | 11.9 |
| DLab.v3+ [11] | ResNeSt-200          | 48.4        | -           | 88M     | 1381G | 8.1  |
| SETR [81]     | T-Large <sup>‡</sup> | 50.3        | 61.7        | 308M    | -     | -    |
| UperNet       | DeiT-S <sup>†</sup>  | 44.0        | -           | 52M     | 1099G | 16.2 |
| UperNet       | Swin-T               | 46.1        | -           | 60M     | 945G  | 18.5 |
| UperNet       | Swin-S               | 49.3        | -           | 81M     | 1038G | 15.2 |
| UperNet       | Swin-B <sup>‡</sup>  | 51.6        | -           | 121M    | 1841G | 8.7  |
| UperNet       | Swin-L <sup>‡</sup>  | <b>53.5</b> | <b>62.8</b> | 234M    | 3230G | 6.2  |



# References

- Transformer
  - A. Vaswani et al.: Attention is All you Need, in NIPS 2017.
- ViT
  - A. Dosovitskiy et al.: An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale, in ICLR 2021.
- DeiT
  - H. Touvron et al.: Training Data-efficient Image Transformers & Distillation through Attention, in ICML 2021.
- Swin Transformer
  - Z. Liu et al.: Swin Transformer: Hierarchical Vision Transformer using Shifted Windows, in ICCV 2021.



# Thank You for Your Attention!

THANK YOU FOR YOUR ATTENTION!

**Yen-Yu Lin (林彥宇)**

Email: [lin@cs.nycu.edu.tw](mailto:lin@cs.nycu.edu.tw)

URL: <https://www.cs.nycu.edu.tw/members/detail/lin>

